# BEYOND THE BLACK BOX: DISENTANGLED AND CONTROLLABLE GENERATIVE MODELS FOR MEDICAL IMAGING

AXEL MARTIN ROMERO ANTOLIN


PhD in Artificial Intelligence
Facultat d'Informàtica de Barcelona (FIB)
Universitat Politècnica de Catalunya (UPC)

January 2029 – version 4.2

Axel Martin Romero Antolin: *Beyond the Black Box: Disentangled and Controllable Generative Models for Medical Imaging*, PhD in Artificial Intelligence, © January 2029

SUPERVISORS:
Prof. Javier Béjar Alonso

LOCATION:
Barcelona, Spain

TIME FRAME:
January 2029

# CONTENTS

# LIST OF FIGURES

# LIST OF TABLES



# NOTATION

- **Scalars** are denoted by lowercase letters, e.g. $x \in \mathbb{R}$.
- **Vectors** are denoted by bold lowercase letters, e.g. $\mathbf{x} \in \mathbb{R}^n$.
- **Matrices** are denoted by bold uppercase letters, e.g. $\mathbf{X} \in \mathbb{R}^{m \times n}$.
- **Sets** are denoted by calligraphic uppercase letters, e.g. $\mathcal{X}$.
- $\mathbf{X}^\top$ denotes the transpose of a matrix $\mathbf{X}$.
- $\mathbf{I}$ denotes the identity matrix.
- $\mathbb{E}_{\mathbf{x}\sim p}[f(\mathbf{x})]$ denotes the expected value of a function $f(\mathbf{x})$ with respect to a probability distribution $p$ over $\mathbf{x}$.
- $\mathrm{Var}_{\mathbf{x}\sim p}[f(\mathbf{x})]$ denotes the variance of a function $f(\mathbf{x})$ with respect to a probability distribution $p$ over $\mathbf{x}$.
- $\nabla_\theta f(\theta)$ denotes the gradient of a function $f(\theta)$ with respect to the parameters $\theta$.

# 1 INTRODUCTION

In recent years, generative artificial intelligence has fundamentally transformed the landscape of synthetic data generation. Early breakthroughs with Generative Adversarial Networks (GANs) and Variational Autoencoders (VAEs) demonstrated the potential of neural networks to learn and sample from complex data distributions. More recently, the emergence of denoising and score-based Diffusion Models and Flow Matching frameworks has set a new standard, achieving unprecedented fidelity and training stability when synthesizing highly realistic, high-dimensional data like images, which is the main focus of this thesis.
However, as these powerful models are increasingly used by the community and the industry, it has become clear that their raw generative capabilities are not sufficient for all applications. For high-risk domains such as healthcare, their inherently opaque, "black box" nature poses significant challenges for the deployment. This thesis aims to bridge the gap between raw generative power and clinical reliability. The primary focus of this research is to develop new methods to enhance the interpretability, disentanglement, and compositionality of state-of-the-art generative frameworks, such as Flow Matching and Diffusion models, for the generation of safe and reliable synthetic medical images. By establishing a deep theoretical understanding of the underlying principles of these models and the concepts of disentanglement and compositionality, this work identifies their current structural weaknesses and advantages and proposes solutions to overcome the complexities to be able to generate high-quality synthetic medical images to address the severe scarcity of annotated medical datasets.

The development and validation of robust medical AI systems require massive amounts of diverse, high-quality, and richly annotated data. However, the acquisition of such datasets is chronically hindered by strict privacy regulations, high annotation costs, and the inherent rarity of certain pathologies. While synthetic data generation offers a promising solution to this bottleneck, current standard generative models often lack the precise control necessary for clinical applications.
When a generative model operates as a "black box," users cannot reliably predict how specific anatomical structures or pathological features will be rendered. The widespread deployment of medical AI hinges fundamentally on the concept of human-AI trust. However,

as recent frameworks on AI trustworthiness emphasize, simply providing mathematical interpretability—trying to understand how the model works internally—is neither necessary nor sufficient for establishing this trust [34]. Instead, human trust in AI is built through interaction. Users must be able to run the model at will and probe its boundaries to observe predictable outcomes, generating empirical "behavior certificates" that prove the reliability of the model under specific constraints.
Therefore, to be truly safe and useful in a medical context, synthetic generation must prioritize this interactive capability through disentanglement. It must allow a clinician or researcher to alter a single semantic feature, such as the size, severity, or location of a tumor, and immediately observe the response of the model without the surrounding healthy anatomy becoming distorted. The motivation for this research stems from the critical need to facilitate this interactive trust. By developing frameworks where different semantic components can be independently manipulated and recombined, this thesis strives to transition generative models from static "black boxes" into highly interactive, trustworthy tools that users can rigorously test, understand through behavior, and safely use to simulate rare clinical scenarios.

To achieve the overarching goal of developing highly controllable and disentangled generative models for medical imaging, this research is driven by the following specific objectives:

- **Theoretical Grounding**: Acquire an in-depth theoretical understanding of diffusion and flow-based generative models, with a specific focus on their probabilistic foundations, interpolation strategies, and current structural limitations.
- **Comprehensive Literature Review**: Conduct a thorough analysis of existing literature surrounding control mechanisms and compositional frameworks in generative modeling, highlighting gaps in their application to the medical domain.
- **Baseline Benchmarking**: Reproduce and systematically benchmark baseline models, including Denoising Diffusion Probabilistic Models (DDPMs) and Flow Matching models, utilizing public and synthetic medical datasets to quantitatively evaluate current limitations in controllability and fidelity.
- **Design of Novel Control Strategies**: Propose and implement novel architectural modifications and training mechanisms, such as semantic conditioning, class guidance, and spatial control, to enforce strict user-defined constraints on the generative process.
- **Compositional Generation**: Develop compositional generation techniques that enable modular synthesis, ensuring that distinct

semantic components (e.g., pathology presence and anatomical structure) can be independently disentangled, manipulated, and seamlessly recombined.

- **Clinical Validation and Evaluation**: Apply the proposed models to real-world medical datasets, rigorously evaluating the methods through both quantitative metrics (e.g., FID, precision-recall, segmentation overlap) and qualitative assessments by clinical experts to ensure the utility, diversity, and realism of the generated data.
- **Dissemination and Open Science**: Contribute to the broader research community by disseminating findings through peer-reviewed publications and providing open-source implementations of the developed frameworks to ensure transparency and reproducibility.

# Part I

# FOUNDATIONS AND LITERATURE REVIEW

# 2

# DEEP GENERATIVE MODELS

Let's start with the definition of our dataset $\mathcal{D}$, consisting of $N \geqslant 1$ data points, where the instances are assumed to be independently and identically distributed. Assume $\mathbf{x}$ represents the set of observed variables (images in our case), which is a random sample from an unknown underlying process, whose true distribution $p^*(\mathbf{x})$ is unknown. The idea is to approximate the real process with a model $p_\theta(\mathbf{x})$ with parameters $\theta$. Learning has the aim to find a value of parameters $\theta$ such that the probability distribution of the model, $p_\theta(\mathbf{x})$, approximates the true distribution for any observed set of variables, i.e. $p_\theta(\mathbf{x}) \approx p^*(\mathbf{x})$.

Deep Generative Models (DGMs) are neural networks, which are a flexible way to parameterise distributions, that learn $p_\theta(\mathbf{x})$, which is commonly referred to as a generative model. Once the model is available, we can generate new samples using sampling methods and compute the likelihood of any given data point $\mathbf{x}'$ by evaluating $p_\theta(\mathbf{x}')$.

## 2.1 TRAINING OF DGM

The general idea is to learn the parameters $\theta$ by minimizing the discrepancy between real and predicted data distribution:

$$\theta^* = \arg\min_\theta \mathcal{D}(p_{data}, p_\theta).$$

Because $p_{data}$ is unknown, we have to choose a discrepancy function that allows efficient estimation from samples from $p_{data}$.

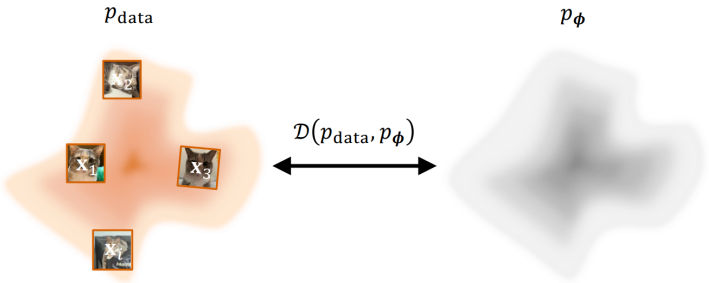



Figure 1: Visualization of the training of DGMs where we minimize a discrepancy measure between the real and unknown data distribution and the approximated one, using a set of i.i.d. samples.[22]

The most common choice is the Kullback-Leibler divergence defined as

$$\begin{aligned} \mathcal{D}_{KL}(p_{data}\|p_\theta) = \int p_{data}(\mathbf{x}) \log \frac{p_{data}(\mathbf{x})}{p_\theta(\mathbf{x})} d\mathbf{x} = \\ \mathbb{E}_{\mathbf{x}\sim p_{data}} \left[\log p_{data}(\mathbf{x}) - \log p_\theta(\mathbf{x})\right], \end{aligned} \tag{1}$$

where, taking into account only the part that depends on the parameter $\theta$, minimizing the KL divergence is equivalent to performing MLE:

$$\min_\theta \mathcal{D}_{KL}(p_{data}\|p_\theta) \iff \max_\theta \mathbb{E}_{\mathbf{x}\sim p_{data}} \left[\log p_\theta(\mathbf{x})\right].$$

In practice, we do not have access to the data distribution, only to some i.i.d. samples $\{\mathbf{x}^{(i)}\}_{i=1}^N \sim p_{data}$, which are used to obtain the empirical estimation of the MSE

$$\mathcal{L}_{MLE}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(\mathbf{x}^{(i)}).$$

There are some challenges in the use of neural networks to parameterize the probability distribution $p_{data}$. We can define a model that produces a real scalar $E_\theta(\mathbf{x}) \in \mathbb{R}$ for input $\mathbf{x}$. To interpret the output as a valid probability density, it has to satisfy the following conditions:

- **Non-Negativity**: $p_\theta(\mathbf{x}) \geqslant 0$ for all $\mathbf{x}$ in the domain. The standard positive function used to guarantee non-negativity is the exponential function, which produces an unnormalized density $\tilde{p}_\theta(\mathbf{x}) = \exp(E_\theta(\mathbf{x}))$.
- **Normalization**: The integral over the entire domain must equal one. To create a valid probability density, we must divide it by its integral over the entire space

  $$p_\theta(\mathbf{x}) = \frac{\tilde{p}_\theta(\mathbf{x})}{Z(\theta)},$$

  where the denominator is the partition function defined as

  $$Z(\theta) = \int \tilde{p}_\theta(\mathbf{x}')d\mathbf{x}'.$$

The partition function introduces a computational challenge in the learning of probabilistic models, especially for high-dimensional problems. This intractability is a central problem that motivates the development of many different families of deep generative models, which will be explained in the following chapters.

# 3

# VARIATIONAL AUTOENCODERS

The core initial idea for the understanding of the Variational Autoencoders (VAEs) is the concept of **Autoencoders** (AEs). This are a type of model that contains two elements, the encoder and the decoder. The encoder is a function that maps the input data $\mathbf{x} \in \mathbb{R}^d$ to a low-dimensional representation $\mathbf{z} \in \mathbb{R}^k$, which is usually much more smaller than the input dimension, $k << d$. This representations live in a space with a more compact and interpretable structure, called the latent space. Then, the decoder is a function that maps the latent representation $\mathbf{z}$ back to the input space, trying to reconstruct the original data point $\mathbf{x}$.
Once we have an autoencoder model trained, a natural question is how to use it as a generative model. One idea is to randomly sample a point $\mathbf{z}$ from the latent space and then pass it through the decoder to get a synthetic data point $\mathbf{x}'$. This method will not produce realistic samples because the latent space is disorganized and irregular. Another option is to encode and image and sample neighboring points in the latent space that could generate similar samples, but the reality is that the output are not meaningful samples. This shows how poorly structured the latent space is. The aim of VAEs is to transform the latent space of an autoencoder into a well-structured space that follows a prior distribution, which allows us to sample from it and generate realistic samples.

It is also important to introduce the concept of latent variable models. This type of variables are part of the model but are not observed in the data, usually denoted as $\mathbf{z}$, which are the hidden factors of variation. The goal of latent variable models is to learn the joint distribution $p_\theta(\mathbf{x}, \mathbf{z})$ of the observed data $\mathbf{x}$ and the latent variables $\mathbf{z}$. The marginal likelihood, model evidence or marginal distribution over the observed data is given by:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z}. \tag{2}$$

When we use neural networks to parameterize the latent variable models we call them **Deep Latent Variable Models** (DLVMs). The simple and most common DLVM is given by the following factorization:

$$p_\theta(\mathbf{x}, \mathbf{z}) = p(\mathbf{z}) p_\theta(\mathbf{x}|\mathbf{z}), \tag{3}$$

where $p(\mathbf{z})$ is the *prior distribution* over the latent variables and $p_\theta(\mathbf{x}|\mathbf{z})$ is the likelihood of the data given the latent variables, also called the

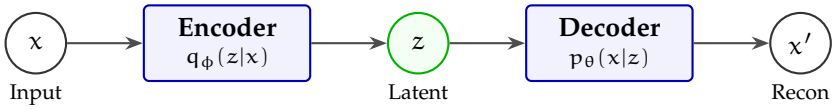



Figure 2: Simplified Variational Autoencoder (VAE) architecture mapping an input $x$ to a latent representation $z$, and decoding it to a reconstruction $x'$.

*generator distribution*. The joint distribution $p_\theta(\mathbf{x}, \mathbf{z})$ is related through the Bayes' rule to the following posterior distribution:

$$p_\theta(\mathbf{z}|\mathbf{x}) = \frac{p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})}{p_\theta(\mathbf{x})}, \tag{4}$$

where we have an intractability problem due to the marginal likelihood $p_\theta(\mathbf{x})$ which is given by an integral over the latent variables (Equation 2). To overcome this problem we can use variational inference (VI) which is a technique to approximate the posterior distribution $p_\theta(\mathbf{z}|\mathbf{x})$ with a simpler distribution $q_\phi(\mathbf{z}|\mathbf{x})$ that is parameterized by $\phi$.

This is the main idea behind **Variational Autoencoders** (VAEs) [17] which are a type of DLVMs that use VI to learn the latent variable model. It defines a parametric inference model $q_\phi(\mathbf{z}|\mathbf{x})$, also called an *encoder*, where $\phi$ are the parameters. The idea is to optimize the parameters such that:

$$q_\phi(\mathbf{z}|\mathbf{x}) \approx p_\theta(\mathbf{z}|\mathbf{x}). \tag{5}$$

This VAE formulation allows us to transform an autoencoder model with an unstructured latent space into a generative model with a latent space that follows a prior distribution, typically a Gaussian distribution $p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, which is easy to sample from.

## 3.1 TRAINING OF VAES

For the training of the VAEs we can not directly optimize the marginal likelihood $p_\theta(\mathbf{x})$ due to the intractability problem, but we can optimize a lower bound of the marginal likelihood called the **Evidence Lower Bound** (ELBO). Let's derive the log-likelihood of the data

point $\mathbf{x}$ and see how we can get the ELBO using the Jensen's inequality [16]:

$$\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}, \mathbf{z}) d\mathbf{z} \tag{6}$$

$$= \log \int q_\phi(\mathbf{z}|\mathbf{x}) \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} d\mathbf{z} \tag{7}$$

$$= \log \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{8}$$

$$\geqslant \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p_\theta(\mathbf{x}, \mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{9}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p_\theta(\mathbf{x}|\mathbf{z}) p(\mathbf{z})}{q_\phi(\mathbf{z}|\mathbf{x})} \right] \tag{10}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p_\theta(\mathbf{x}|\mathbf{z})}{\frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})}} \right] \tag{11}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log p_\theta(\mathbf{x}|\mathbf{z}) - \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right] \tag{12}$$

$$= \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction}} - \underbrace{\mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} \left[ \log \frac{q_\phi(\mathbf{z}|\mathbf{x})}{p(\mathbf{z})} \right]}_{\text{KL Divergence}} \tag{13}$$

We can see that for any data point $\mathbf{x}$, the log-likelihood satisfies:

$$\log p_\theta(\mathbf{x}) \geqslant \mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) \tag{14}$$

$$= \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z})). \tag{15}$$

If we maximize the previous loss with respect to the parameters $\theta$ and $\phi$, we are optimizing:

- **Reconstruction**: the first term encourage to maximize the likelihood $p_\theta(\mathbf{x}|\mathbf{z})$, which corresponds to the reconstruction of the data point $\mathbf{x}$ from the latent variables $\mathbf{z}$. Optimizing this term alone risks memorizing the training data, so we need extra regularization.
- **Latent KL Regularization**: the second term encourages the encoder distribution $q_\phi(\mathbf{z}|\mathbf{x})$ to be close to the prior distribution $p(\mathbf{z})$, which is easy to sample from.

Once we have the definition of the training objective, we can use stochastic gradient descent (SGD) to perform joint optimization w.r.t. all parameters $\theta$ and $\phi$. However, we have a problem with the first term of the ELBO because we can not backpropagate through the sampling process of $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$, as we can see in the left part of Figure 3. To solve this problem we can use the **reparameterization trick** which consists in expressing the sampling process as a deterministic function of the parameters and some noise. For example, if we assume

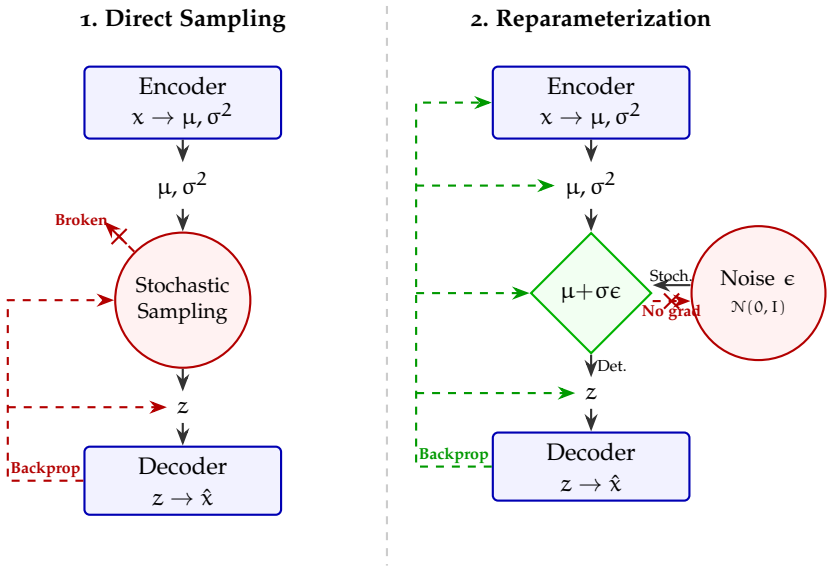



Figure 3: Reparameterization Trick in VAEs

that $q_\phi(\mathbf{z}|\mathbf{x})$ is a Gaussian distribution with mean $\mu$ and variance $\sigma^2$, we can sample from it as follows:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I). \tag{16}$$

This way, we can backpropagate through the deterministic function and optimize the parameters $\phi$ of the encoder. This is shown in the right part of Figure 3.

## 3.2 GAUSSIAN VAES

The common choice for both the encoder and the decoder distributions in VAEs is the Gaussian distribution. This is because it allows us to use the reparameterization trick and also because it has a closed-form expression for the KL divergence.
The **encoder** $q_\phi(\mathbf{z}|\mathbf{x})$ is modelled as a Gaussian distribution defined as:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}\left(\mathbf{z}; \mu_\phi(\mathbf{x}), \text{diag}(\sigma^2_\phi(\mathbf{x}))\right), \tag{17}$$

where $\mu_\phi(\mathbf{x})$ and $\sigma^2_\phi(\mathbf{x})$ are the mean and variance of the Gaussian distribution, which are parameterized by a neural network with parameters $\phi$.
The **decoder** $p_\theta(\mathbf{x}|\mathbf{z})$ is also modelled as a Gaussian distribution defined as:

$$p_\theta(\mathbf{x}|\mathbf{z}) = \mathcal{N}\left(\mathbf{x}; \mu_\theta(\mathbf{z}), \sigma^2\mathbf{I}\right), \tag{18}$$

where $\mu_\theta(\mathbf{z})$ is the mean of the Gaussian distribution, which is parameterized by a neural network with parameters $\theta$, and $\sigma^2\mathbf{I}$ is the

covariance matrix, which is usually fixed to be a diagonal matrix with constant variance.

Under this Gaussian assumption, we can obtain a closed-form expression for the ELBO loss function. The reconstruction term can be expressed as:

$$\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\|\mathbf{x}-\mu_\theta(\mathbf{z})\|^2\right] + C, \quad (19)$$

where C is a constant that does not depend on the parameters. The KL divergence term can be expressed as:

$$\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) = \frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right), \quad (20)$$

where k is the dimension of the latent space. Therefore, the ELBO loss function can be expressed as:

$$\mathcal{L}_{ELBO}(\theta,\phi;\mathbf{x}) = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\|\mathbf{x}-\mu_\theta(\mathbf{z})\|^2\right] \quad (21)$$

$$-\frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right) + C. \quad (22)$$

Despite their mathematical elegance and continuous latent spaces, standard Gaussian Variational Autoencoders notoriously suffer from generating blurry images. This limitation fundamentally stems from the VAE objective function and its underlying statistical assumptions. The problem can be attributed to four primary factors:

- **MSE Loss**: In a standard Gaussian VAE with a fixed variance $\sigma^2\mathbf{I}$, maximizing the likelihood $p_\theta(\mathbf{x}|\mathbf{z})$ is mathematically equivalent to minimizing the $\mathcal{L}_2$ distance between the input $\mathbf{x}$ and the reconstruction $\hat{\mathbf{x}}$:

  $$\log p_\theta(\mathbf{x}|\mathbf{z}) \propto -\|\mathbf{x}-\mu_\theta(\mathbf{z})\|_2^2 \quad (23)$$

  The $\mathcal{L}_2$ loss heavily penalizes large pixel-wise deviations. When the model is uncertain about the exact location of high-frequency details (such as sharp edges), it outputs the statistical mean of all plausible variations to minimize the overall penalty. This phenomenon, known as *mode averaging*, results in safe, blurry predictions rather than rendering sharp, distinct features.

- **Information Bottleneck**: The ELBO includes a regularization term that minimizes the KL divergence between the approximate posterior and the prior:

  $$\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) \quad (24)$$

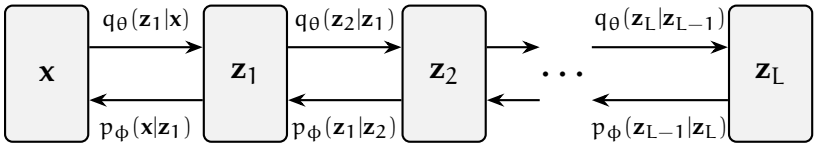



Figure 4: Hierarchical VAE architecture with L layers of latent variables. Each layer captures different levels of abstraction, allowing for richer representations and improved generation quality.

This term acts as an information bottleneck, forcing the encoded latent representations to closely match the simple prior, typically $\mathcal{N}(\mathbf{0}, \mathbf{I})$. Consequently, the encoder discards complex, high-frequency spatial features that are difficult to compress, prioritizing global structures and low-frequency components instead.

- **Posterior Collapse**: Furthermore, consistent with findings in [2] and [37], stochastic optimization using the unmodified lower bound objective frequently gets stuck in an undesirable stable equilibrium. At the onset of training, the reconstruction likelihood term $\log p_\theta(\mathbf{x}|\mathbf{z})$ is relatively weak. Consequently, an initially attractive state for the network is to aggressively minimize the latent cost, driving the approximate posterior to match the prior, $q_\phi(\mathbf{z}|\mathbf{x}) \approx p(\mathbf{z})$. This results in a stable equilibrium—often termed *posterior collapse*—from which the model struggles to escape, yielding uninformative latent representations and contributing to poor generation quality. To mitigate this, a common solution proposed in [2, 37] is to implement an optimization schedule (KL annealing) where the weight of the latent cost $\mathcal{D}_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ is slowly annealed from 0 to 1 over multiple epochs.

## 3.3 DENOISING DIFFUSION PROBABILISTIC MODELS

One important technique developed to enhance the performance of VAEs and overcome the main limitations was **Hierarchical Variational Autoencoders** (HVAEs) [38] which are a type of VAE that uses a hierarchical structure in the latent space. This hierarchical structure allows the model to capture more complex and high-frequency details in the data, which can help to reduce the blurriness of the generated images. A visualization of the architecture of HVAEs is shown in Figure 4. In this architecture, we have L layers of latent variables $\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_L$, where each layer captures different levels of abstraction.
In this setup, we can define the following factorization of the joint distribution:

$$p_\theta(\mathbf{x}, \mathbf{z}_{1:L}) = p_\theta(\mathbf{x}|\mathbf{z}_1) \prod_{i=2}^{L} p_\theta(\mathbf{z}_{i-1}|\mathbf{z}_i) p(\mathbf{z}_L), \tag{25}$$

and the marginal distribution of the data point $\mathbf{x}$ can be expressed as:

$$p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x}, \mathbf{z}_{1:L}) d\mathbf{z}_{1:L}, \tag{26}$$

where the generation is performed progressively from the latent variable $\mathbf{z}_L$, which follows the prio distribution $p(\mathbf{z}_L)$, to the data point $\mathbf{x}$, which is generated from the latent variable $\mathbf{z}_1$. The inference model can be defined as:

$$q_\phi(\mathbf{z}_{1:L}|\mathbf{x}) = q_\phi(\mathbf{z}_1|\mathbf{x}) \prod_{i=2}^{L} q_\phi(\mathbf{z}_i|\mathbf{z}_{i-1}), \tag{27}$$

where the inference is performed progressively from the data point $\mathbf{x}$ to the latent variable $\mathbf{z}_L$.
The ELBO loss is very similar to the one of the standard VAE, but we have to consider all the layers of latent variables:

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}_{1:L} \sim q_\phi(\mathbf{z}_{1:L}|\mathbf{x})} \left[ \log \frac{p_\theta(\mathbf{x}, \mathbf{z}_{1:L})}{q_\phi(\mathbf{z}_{1:L}|\mathbf{x})} \right], \tag{28}$$

which can also be decomposed into the reconstruction term and the KL divergence term.

This method of deep, stacked layers revolutionized the field of generative modeling and is the basis for many state-of-the-art models. The training of HVAEs present several challenges because the encoder and decoder must be trained jointly and learning becomes unstable. A clever twist to solve the main problems and maintain the hierarchical structure is the **Denoising Diffusion Probabilistic Models** (DDPMs) [9, 36], the next step in generative modeling, that can be defined with the following stochastic processes.

**Forward Pass**. This is a fixed encoder (not learnable) that progressively adds noise to the data point $\mathbf{x}$ via a transition kernel $p(\mathbf{x}_t|\mathbf{x}_{t-1})$ until we get pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The fixed Gaussian transition kernel is defined as:

$$p(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t \mathbf{I}), \tag{29}$$

where $\mathbf{x}_0 \sim p_{data}$, $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and the sequence $\{\beta_t\}_{t=1}^T \in (0, 1)$ is the noise schedule, which controls the amount of noise added at each step, as we can clearly see in the following formulation:

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1-\alpha_t}\epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{30}$$

where $\alpha_t = 1 - \beta_t$. Another important formulation of the forward pass can be achieve by recursively applying the transition kernel (Equation 29), where we achieve a closed-form expression for the distribution of noisy samples at step $t$ given the original data:

$$p(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}), \tag{31}$$

where $\bar{\alpha}_t = \prod_{s=1}^{T} \alpha_s$. This formulation is very useful because it allows us to sample from the forward process at any time step $t$ without having to recursively apply the transition kernel $t$ times. We can also define the sampling of $\mathbf{x}_t$ as in Equation 30 as follows:

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \tag{32}$$

**Reverse Pass**: This is the decoder, where a neural network is trained to reverse the noising process by learning the parameterized distribution $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. If we start from a pure noise sample, we can iteratively apply the reverse process to generate a data point $\mathbf{x}_0$ that resembles the training data, following a Markov chain. The training goal is to approximate the unknown reverse transition kernel $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$ with a learnable parametric model $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$, using the KL divergence as the training objective:

$$\mathbb{E}_{\mathbf{x}_t \sim p_t}\left[\mathcal{D}_{KL}(p(\mathbf{x}_{t-1}|\mathbf{x}_t)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))\right]. \tag{33}$$

We can express the unknown reverse transition kernel $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$ in terms of the forward process using Bayes' rule:

$$p(\mathbf{x}_{t-1}|\mathbf{x}_t) = \frac{p(\mathbf{x}_t|\mathbf{x}_{t-1})p_{t-1}(\mathbf{x}_{t-1})}{p_t(\mathbf{x}_t)}, \tag{34}$$

where the marginal distribution $p_t(\mathbf{x}_t)$ can be expressed as:

$$p_t(\mathbf{x}_t) = \int p(\mathbf{x}_t|\mathbf{x}_0)p_{data}(\mathbf{x}_0)d\mathbf{x}_0. \tag{35}$$

Since $p_{data}$ is unknown, we can not compute the marginal distribution $p_t(\mathbf{x}_t)$ thus the computation of the target distribution $p(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is intractable. A trick to solve this problem is to condition the reverse transition kernel on the original data point $\mathbf{x}_0$. Now we can define the target distribution as follows:

$$p(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{p(\mathbf{x}_t|\mathbf{x}_{t-1})p_{t-1}(\mathbf{x}_{t-1}|\mathbf{x}_0)}{p_t(\mathbf{x}_t|\mathbf{x}_0)}, \tag{36}$$

which allows us to have a tractable expression and to derive a tractable objective equivalent to the previous intractable KL divergence in Equation 33. As a result, $p(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ is a Gaussian distribution with mean and variance that can be computed in closed-form:

$$p(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t), \sigma^2(t)\mathbf{I}), \tag{37}$$

where

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_0 + \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t, \quad \sigma^2(t) = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t. \tag{38}$$

Finally, DDPM assumes that each reverse transition kernel $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is a Gaussian distribution with mean $\mu_\theta(\mathbf{x}_t, t)$ and variance $\sigma^2(t)\mathbf{I}$, where the variance is fixed and the mean is parameterized by a neural network with parameters $\theta$. It can be defined as

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \sigma^2(t)\mathbf{I}). \tag{39}$$

Using the previous conditional derivation of the unknown reverse transition kernel, we can define a closed-form expression for the KL divergence training loss:

$$\mathcal{L}(\mathbf{x}_0; \theta) = \mathbb{E}_{t,\mathbf{x}_0}\left[\frac{1}{2\sigma^2(t)}\|\mu_\theta(\mathbf{x}_t, t) - \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t)\|^2 + C\right]. \tag{40}$$

where C is a constant that does not depend on the parameters. This loss can be interpreted as a weighted $\mathcal{L}_2$ loss between the predicted mean $\mu_\theta(\mathbf{x}_t, t)$ and the true mean $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t)$ of the reverse transition kernel.
In practice, we do not use the loss of Equation 40 directly. If we combine the definition of the true mean $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t)$ in Equation 38 with Equation 32, we can obtain two different parameterizations of the mean that can be used to derive two different but equivalent training losses that are easier to optimize.

- $\epsilon$-Prediction. From Equation 32 we obtain $\mathbf{x}_0 = \frac{1}{\sqrt{\bar{\alpha}_t}}(\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\epsilon)$ and we can substitute this expression in the definition of the true mean $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t)$ to obtain the following expression:

  $$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right), \tag{41}$$

  which can also be applied to parameterized mean using now a neural network that predicts the noise $\epsilon_\theta(\mathbf{x}_t, t)$ instead of the mean $\mu_\theta(\mathbf{x}_t, t)$:

  $$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(\mathbf{x}_t, t)\right). \tag{42}$$

  We can define a new loss function based on the noise prediction as follows:

  $$\mathcal{L}(\mathbf{x}_0; \theta) = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[\frac{\beta_t^2}{2\sigma^2(t)\alpha_t(1-\bar{\alpha}_t)}\|\epsilon_\theta(\mathbf{x}_t, t) - \epsilon\|^2\right], \tag{43}$$

  where a simple version of this loss, where the weighting factor is removed, is commonly used in practice for the good performance and stability of the training process found in [9]. The final loss can be expressed as:

  $$\mathcal{L}(\mathbf{x}_0; \theta) = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[\|\epsilon_\theta(\mathbf{x}_t, t) - \epsilon\|^2\right]. \tag{44}$$

- $\mathbf{x}_0$-Prediction: With the definition of the true mean $\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0, t)$ (Equation 38) we can obtain a new parameterization of the mean by using a neural network that predicts the original data point $\mathbf{x}_0$ instead of the mean $\mu_\theta(\mathbf{x}_t, t)$:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}\mathbf{x}_\theta(\mathbf{x}_t, t) + \frac{(1-\bar{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\bar{\alpha}_t}\mathbf{x}_t, \tag{45}$$

  which, as in the previous case, can be used to derive a new loss function based on the original data point prediction as follows:

$$\mathcal{L}(\mathbf{x}_0; \theta) = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[\omega_t \|\mathbf{x}_\theta(\mathbf{x}_t, t) - \mathbf{x}_0\|^2\right]. \tag{46}$$

  where $\omega_t$ is a weighting factor that depends on the noise schedule and the variance of the reverse transition kernel, which can be ignored in practice.

**Summary of Key Equations: VAEs and DDPMs**

This box summarizes the main mathematical equations of VAEs and DDPMs:

**1. Training Objective of VAEs:**

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - \mathcal{D}_{\text{KL}}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})). \tag{47}$$

**2. Training Objective of Gaussian VAEs:**

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = -\frac{1}{2\sigma^2}\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[||\mathbf{x} - \mu_\theta(\mathbf{z})||^2\right] \tag{48}$$

$$-\frac{1}{2}\sum_{i=1}^{k}\left(\sigma^2_{\phi,i}(\mathbf{x}) + \mu^2_{\phi,i}(\mathbf{x}) - 1 - \log\sigma^2_{\phi,i}(\mathbf{x})\right) + C. \tag{49}$$

**3. Forward Pass in DDPMs:**

$$p(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \tag{50}$$

$$p(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1-\bar{\alpha}_t)\mathbf{I}) \tag{51}$$

**4. $\epsilon$-Prediction loss in DDPMs:**

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[||\epsilon_\theta(\mathbf{x}_t, t) - \epsilon||^2\right]. \tag{52}$$

**5. $\mathbf{x}_0$-Prediction loss in DDPMs:**

$$\mathcal{L}(\theta) = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\left[\omega_t||\mathbf{x}_\theta(\mathbf{x}_t, t) - \mathbf{x}_0||^2\right]. \tag{53}$$

# 4

# ENERGY-BASED MODELS

EBMs.

**Summary of Key Equations: Energy-Based Models**

This box summarizes the main mathematical equations of EBMs:

**1. Training Objective of VAEs:**

$$\mathcal{L}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{z \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) || p_\theta(\mathbf{z})) \quad (54)$$

**2. Reparameterization Trick:**

$$\mathbf{x}_{\text{synthetic}} = G(\mathbf{z}_{\text{anatomy}} \oplus \mathbf{z}_{\text{pathology}} \mid \mathbf{y}) \quad (55)$$

**Note:** Ensure latent dimensions $z_i$ and $z_j$ maintain near-zero mutual information to preserve explainability.

# 5 NORMALIZING FLOWS

Normalizing Flows.

# 6 DISENTANGLEMENT

**Disentanglement Representation Learning** (DRL) is a learning paradigm in which models are designed to extract and disentangle the hidden factors of variation in the data, $\mathbf{z} \in \mathbb{R}^c$, given an input observation $\mathbf{x} \in \mathbb{R}^d$. These generative factors are often called latent variables. The formal definition of DRL proposed in [1] is that "Disentangled representation should separate the distinct, independent and informative generative factors of variation in the data. Single latent variables are sensitive to changes in single underlying generative factors, while being relatively invariant to changes in other factors". An important property is that latent variables are statistically independent. This organisation of the latent space into explainable semantic representations improves the following properties of a generative model:

- **Explanability**: the meaningful and independent representations are aligned semantically with latent generative factors.
- **Generalizability**: the representations are well separated from the original entangled input, which improves the generalization ability for the generation of new samples.
- **Controllability**: DRL allows for controllable generation by manipulating the learned disentangled representations of the latent space.

There are several disentanglement methods in the current literature, which can be grouped based on the generative model they use [39], which will be explained in the following sections.
One important aspect of DRL is the

## 6.1 VAE-BASED MODELS

In Section – we defined the Variational Autoencoder (VAE) model, where the loss function is defined in Equation –. This type of model has the potential to learn disentangled representations in simple datasets, because of the choose of the prior distribution $p(\mathbf{z})$ as a normal distribution, which imposes independence constraints. In more complex datasets, the disentanglement capability presented in this model is poor.

One of the first methods was **β-VAE** [8], which introduces a β penalty

hyperparameter before the KL term in the ELBO loss. The new training objective is defined as

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - \beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})), \quad (56)$$

where a value of $\beta = 1$ corresponds to the original VAE implementation and larger values of $\beta > 1$ encourage learning more disentangled representations, adding more pressure to the latent bottleneck to be more factorised, while reducing the performance of reconstruction. This shows an important trade-off between reconstruction accuracy and the disentanglement quality of the latent representation.

Several studies have analized this trade-off and the effect of the $\beta$ hyperparameter in the disentanglement performance. First, we can define the marginal posterior distribution as

$$q_\phi(\mathbf{z}) = \mathbb{E}_{p_{data}(\mathbf{x})}\left[q_\phi(\mathbf{z}|\mathbf{x})\right] = \int q_\phi(\mathbf{z}|\mathbf{x}) p_{data}(\mathbf{x}) d\mathbf{x}, \quad (57)$$

where a disentangled representation is achieved when $q_\phi(\mathbf{z})$ is close to the factorial prior $p(\mathbf{z})$, which means we want a factorial distribution $q_\phi(\mathbf{z}) = \prod_j q_\phi(z_j)$. To gain a deep insight about the trade-off, we can break down the KL term of the loss function as proposed in several papers [4, 10]

$$\mathbb{E}_{p_{data}(\mathbf{x})}\left[KL(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))\right] = \underbrace{I(\mathbf{x};\mathbf{z})}_{\text{(i) Mutual Information}} \quad (58)$$

$$+ \underbrace{D_{KL}(q_\phi(\mathbf{z})||\bar{q}_\phi(\mathbf{z}))}_{\text{(ii) Total Correlation}}, \quad (59)$$

$$+ \underbrace{D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z}))}_{\text{(iii) Prior KL}}, \quad (60)$$

where $I(\mathbf{x};\mathbf{z})$ is the mutual information between $\mathbf{x}$ and $\mathbf{z}$, Total Correlation (TC) is a measure of dependence between the variables and the prior KL prevents the latents to deviate too far from the prior. Here we can clearly see that making $\beta$ larger pushes $q_\phi(\mathbf{z})$ towards the factorial prior $p(\mathbf{x})$ and pushes the model to find statistically independent factoes, improving the disentanglement, but reduces the amount of information about $\mathbf{x}$ stored in $\mathbf{z}$, producing a poor reconstruction.

This trade-off between reconstruction and disentanglement is the main motivation for the development of new methods based on the previous decomposition of the KL term. For example, the **DIP-VAE-I** and **DIP-VAE-II** [20] methods proposed a regularization term on the third

element of the KL decomposition. The objective function can be defined as

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{61}$$

$$- \lambda D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z})). \tag{62}$$

This objective function imposes independence on the variational marginal distribution $q_\phi(\mathbf{z})$. To minimize the distance, they force the covariance of $q_\phi(\mathbf{z})$ to be close to the identity matrix. The covariance is defined as

$$\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}] = \mathbb{E}_{p_{data}(\mathbf{x})}\left[\Sigma_\phi(\mathbf{x})\right] + \mathrm{Cov}_{p_{data}(\mathbf{x})}\left[\mu_\phi(\mathbf{x})\right]. \tag{63}$$

The two variations of DIP-VAE differ in the way they penalise the covariance:

$$D_{KL}(q_\phi(\mathbf{z})||p(\mathbf{z})) = \tag{64}$$

$$\underbrace{= \lambda_{od} \sum_{i\neq j} [\mathrm{Cov}_{p_{data}}[\mu_\phi(\mathbf{x})]]^2_{ij} + \lambda_d \sum_i \left(\mathrm{Cov}_{p_{data}}[\mu_\phi(\mathbf{x})]_{ij} - 1\right)^2}_{\text{DIP-VAE-I}}, \tag{65}$$

$$\text{or} \tag{66}$$

$$\underbrace{= \lambda_{od} \sum_{i\neq j} [\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}]]^2_{ij} + \lambda_d \sum_i \left([\mathrm{Cov}_{q_\phi(\mathbf{z})}[\mathbf{z}]]_{ij} - 1\right)^2}_{\text{DIP-VAE-II}}, \tag{67}$$

where $\lambda_{od}$ and $\lambda_d$ are the hyperparameters that control the strength of the regularization.

The next method, **FactorVAE** [14], proposed to only penalises the second component of the KL decomposition, the Total Correlation (TC), which encourage independence in the latent space. The loss function in this method is defined as:

$$\mathcal{L}_{ELBO}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] - D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z})) \tag{68}$$

$$- \gamma D_{KL}(q_\phi(\mathbf{z})||\bar{q}_\phi(\mathbf{z})), \tag{69}$$

where $\bar{q}_\phi(\mathbf{z}) = \prod_j q_\phi(z_j)$. TC measures the dependencies between the variables in the latent representation. Penalising this specific term encourages the latent dimensions to be statistically independent, which is the core mathematical interpretation of disentanglement.
Because the Total Correlation is computationally intractable to evaluate directly over the entire dataset, FactorVAE employs an adversarial training approach. A discriminator network is trained to distinguish between samples drawn from the aggregated posterior $q_\phi(\mathbf{z})$

and samples drawn from the product of its marginals $\prod_j q_\phi(z_j)$, effectively estimating and minimizing the density ratio. This approach achieves better disentanglement than β-VAE while preserving a higher reconstruction quality.

The paper that proposed the decomposition of the KL term [4] also proposed a method called β-**TCVAE**, which penalize all the terms of the KL decomposition independently, with different hyperparameters. The loss function is defined as

$$\mathcal{L}_{\text{ELBO}}(\theta, \phi; \mathbf{x}) = \mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}\left[\log p_\theta(\mathbf{x}|\mathbf{z})\right] \tag{70}$$

$$- \alpha I(\mathbf{x}; \mathbf{z}) - \beta D_{KL}(q_\phi(\mathbf{z})\|\bar{q}_\phi(\mathbf{z})) - \gamma D_{KL}(q_\phi(\mathbf{z})\|p(\mathbf{z})), \tag{71}$$

All the previous methods are unsupervised with the common idea of adding extra regularization terms to the original VAE loss function to encourage disentanglement.

## 6.2 DIFFUSION / FLOW-BASED MODELS

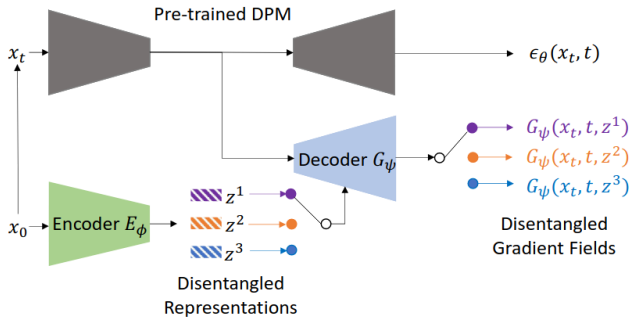
Diffusion and Flow models are a more recent and advanced class of generative models that have drawn a lot of attention in recent years because of their remarkable performance. Disentangled representation learning can also be applied to these types of models with the aim to overcome the important trade-off between reconstruction and disentanglement of the VAE-based methds.

One of the first approaches was DisDiff [42], which learns a disentangled representation and a disentangled gradient field for each generative factor. Assume the data samples are generated by N underlying ground truth factors $f^c$, $c \in \{1, \dots, N\}$. Each factor c follows the distribution $p(f^c)$. We can modify the original score function (Equation –) with the conditioning of the factors using the Bayes' Rule as follows:

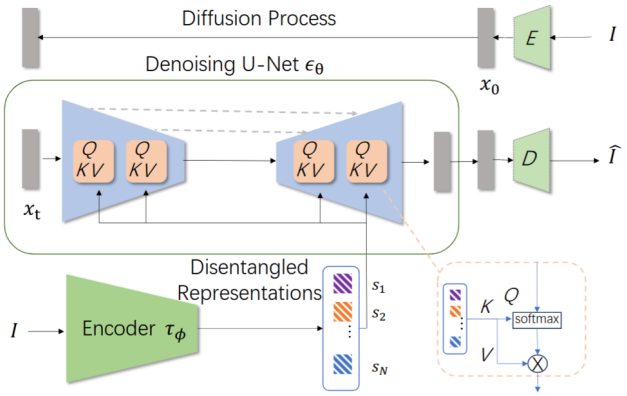
$$\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t|f^c) = \nabla_{\mathbf{x}_t} \log p(f^c|\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t), \tag{72}$$

where we already have the score $\nabla_{\mathbf{x}_t} \log p(\mathbf{x}_t)$ from a pretrained unconditional model and we only have to learn $\nabla_{\mathbf{x}_t} \log p(f^c|\mathbf{x}_t)$. Since we are in an unsupervised setting, we do not know $f^c$. We have to learn a set of representations $\{z^c|c = 1, \dots, N\}$ that must encompass all information of the original samples $\mathbf{x}_0$ and represent the true factors of variations $f^c$. To obtain these representations we use a set of encoders with learnable parameters φ, defined as $E_\phi(\mathbf{x}_0) = \{E^1_\phi(\mathbf{x}_0), \dots E^N_\phi(\mathbf{x}_0)\} = \{z^1, \dots, z^N\}$. Then, assume we have a factor subset $\mathcal{S} \subseteq \mathcal{C}$ with $z^{\mathcal{S}} = \{z^c|c \in \mathcal{S}\}$, the gradient field can be defined as

$$\nabla_{\mathbf{x}_t} \log p(z^{\mathcal{S}}|\mathbf{x}_t) = \sum_{c\in\mathcal{S}} \nabla_{\mathbf{x}_t} \log p(z^c|\mathbf{x}_t). \tag{73}$$




(a) DisDiff Framework




(b) EncDiff Framework

Figure 5: Illustrations of the frameworks for the (a) DisDiff model [42] and the (b) EncDiff model. [41]

A visual representation of this method can be seen in Figure 5(a). Finally, to estimate the gradient fields, they used a network $G_\psi(\mathbf{x}_t, z^c, t)$ and the reconstruction loss is defined as

$$\mathcal{L}_r = \mathbb{E}_{\mathbf{x}_0,t,\epsilon}\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t) + \frac{\sqrt{\alpha_t}\sqrt{1-\bar{\alpha}_t}}{\beta_t}\sigma_t \sum_{c\in\mathcal{C}} G_\psi(\mathbf{x}_t, z^c, t)\|. \quad (74)$$

This loss formulation alone does not guarantee disentanglement among the representations $z^c$. The authors proposed a disentanglement loss that minimizes the upper bound for the mutual information between $z^c$ and $z^j$, where $c \neq j$, defined as

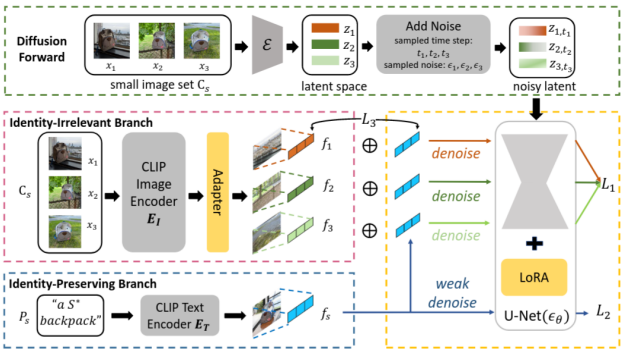
$$\mathcal{L}_{dis} = \min_{i,j,\mathbf{x}_0,\hat{\mathbf{x}}_0^j} \|\hat{z}^{c|j} - \hat{z}^c\| - \|\hat{z}^{c|j} - z^c\| + \|\hat{z}^{j|j} - z^j\|, \quad (75)$$

where $\hat{\mathbf{x}}_0^j$ is conditioned on $z^j$, $\hat{z}^c = E_\phi^c(\hat{\mathbf{x}}_0)$ and $\hat{z}^{c|j} = E_\phi^c(\hat{\mathbf{x}}_0^j)$ where $\hat{\mathbf{x}}_0$ is a sampled data from the pretrained model.

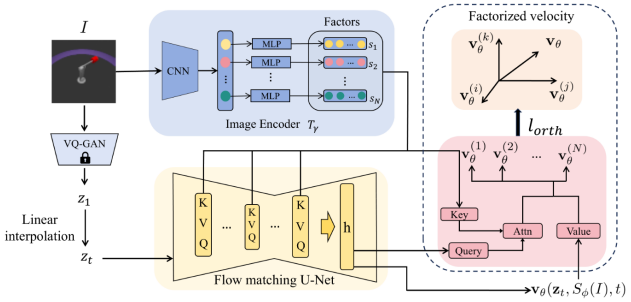
EncDiff [41] is an improvement of the previous method where an image encoder $\tau_\phi$ aims to provide a set of concept tokens $\mathcal{S} = \{s_1, \ldots, s_N\}$. Each dimension of the encoded feature vectors is treated as a disentangled factor and maps each to a vector using a non-shared MLP

layer. Then, based on Latent Diffusion Models [33], the idea is to use the cross-attention mechanism to map the tokens to the intermediate representations of the U-Net model. The general framework of this method is illustrated in Figure 5(b).

DisenBooth [3] After the development of several diffusion-based dis-




(a) DisenBooth Framework




(b) FM Disentanglement Framework

Figure 6: Illustrations of the frameworks for the (a) DisenBooth model [3] and the (b) FM Disentanglement model. [5]

entanglement methods, flow matching with good performance and deterministic formulation has gained attention in this field. In a recent paper, a Flow Matching-Based disentanglement framework [5] was presented that encourages factor-specific, non-overlapping latent transformations and explicit semantic alignment. The pipeline of this method can be seen in Figure 6(b). Given an image $\mathbf{I}$, they encode it into a latent $\mathbf{Z}$ using a pre-trained VQ-GAN encoder, where $\mathbf{z}_1$ represents the latent of an image of the dataset. Then, a factor encoder ($T_\gamma(\mathbf{x})$) is trained to extract a set of N factors. These factors are injected into the flow network via cross-attention, where intermediate features of the network attend to the set of factors. The model outputs a factor-conditioned velocity field $\mathbf{v}_\theta(\mathbf{z}, T_\gamma(\mathbf{x}), t)$. Since the flow matching loss only takes into account the velocity field, it does not encourage disentanglement by default. The next step is to decompose the velocity into N factor-aligned components.

The idea is to decompose the factor-conditioned velocity field as follows:

$$\mathbf{v}_\theta(\mathbf{z}_t, T_\gamma(\mathbf{x}), t) = \sum_{i=1}^{N} \mathbf{v}_\theta^{(i)}(\mathbf{z}_t, T_\gamma(\mathbf{x}), t) \tag{76}$$

where $\mathbf{v}_\theta^{(i)}$ is the component attributed to the $i$-th factor of variation. They add an orthogonality loss to penalise pairwise alignment between factor-specific components and encourage different factors to capture distinct transport directions. For the definition of the different velocity fields, they used output attention. Let $h$ denote the last hidden feature map of the network, we define the queries and keys as:

$$\mathbf{Q} = \mathbf{W}_q h, \quad \mathbf{K}_i = \mathbf{W}_k s_i \tag{77}$$

where $\mathbf{W}_q$ and $\mathbf{W}_k$ are learned projections, $s_i$ a factor, and $d$ the query/key dimensionality. The attention is defined as

$$\mathrm{Attn}_i = \frac{\exp\left(\frac{\mathbf{Q}\mathbf{K}_i^T}{\sqrt{d}}\right)}{\sum_{j=1}^{N} \exp\left(\frac{\mathbf{Q}\mathbf{K}_i^T}{\sqrt{d}}\right)}, \tag{78}$$

which satisfies that $\sum_{i=1}^{N} \mathrm{Attn}_i = 1$ at each spatial location. Then, the factor-specific velocity field is defined as

$$\mathbf{v}_\theta^{(i)}(\mathbf{z}_t, T_\gamma(\mathbf{x}), t) = \mathrm{Attn}_i \odot \mathbf{v}_\theta(\mathbf{z}_t, T_\gamma(\mathbf{x}), t), \tag{79}$$

where $\odot$ denotes the element-wise multiplication. The final training objective is defined as

$$\mathcal{L}(\theta) = \mathcal{L}_{FM}(\theta) + \lambda_{orth}\mathcal{L}_{orth} \tag{80}$$

where $\lambda_{orth}$ controls the strength of the regularization.

## 6.3 METRICS

One of the initial methods for the evaluation of disentanglement representation learning was inspecting the change in reconstruction when traversing one variable in the latent space. This qualitative evaluation is useful, but it is important to have reliable quantitative metrics to measure disentanglement. As shown in the previous sections, significant advances have been made in this field, but the literature still lacks a reliable and unified metric for evaluation. As presented in –, a good metric has to evaluate the separation of interpretable factors, the capture of the diversity of the data, the invariance across changes in unrelated dimensions and the summarisation of the important information efficiently in a disentangled representation.

One of the initial metrics presented in the literature are **Z-diff**[8] and **Z-min Variance**[14], where both of them are supervised, i.e. we need the ground truth factors of variations. The idea is to choose a factor $k$, generate data with this factor fixed and the others varying randomly, obtain the representations through the encoder ($q_\phi(\mathbf{z}|\mathbf{x})$) and take the absolute value of the pairwise differences. In Z-diff, the metric trains a classifier to identify which factor is fixed and reports the accuracy value as the disentanglement metric. On the other hand, metric Z-min Variance shows that the previous one has some problems related to the training of a model and proposes to avoid training a classifier and use a majority-vote classifier with no optimisation hyperparameters.

# 7 COMPOSITIONALITY

Compositionality.

## 7.1 OBJECT-CENTRIC LEARNING

Slot Attention (SA) [25] is a differentiable interface between image features and a set of variables called slots. This method maps a set of $N$ image patches with dimension $D_{input}$, called image features $\mathbf{F} \in \mathbb{R}^{N \times D_{input}}$, to a set of $K$ output vectors of size $D_{slot}$ called slots, $\mathbf{S} \in \mathbb{R}^{K \times D_{slot}}$. Each vector of the slots can describe an object or element in the input image. The process of binding a part of the image to the slots is performed via iterative refinement over $T$ steps, where in each iteration the slots compete via a softmax attention mechanism to explain parts of the input. Slot representations are initialised randomly with a Gaussian distribution.
If we define some learnable linear transformations $k$, $q$ and $v$ to map input features and slots to a common dimension $D$, we can define:

$$\mathbf{A}_{i,j} = \frac{e^{\mathbf{M}_{i,j}}}{\sum_l e^{\mathbf{M}_{i,l}}}, \text{ where } \mathbf{M} = \frac{1}{\sqrt{D}} k(\mathbf{F}) \cdot q(\mathbf{S})^T \in \mathbb{R}^{N \times K} \tag{81}$$

The normalisation in the attention ensures that attention coefficients sum to one for each individual input feature vector. The update for the slots can be defined as:

$$\mathbf{U} = \mathbf{W}^T \cdot v(\mathbf{F}) \in \mathbb{R}^{K \times D} \text{ where } \mathbf{W}_{i,j} = \frac{\mathbf{A}_{i,j}}{\sum_{l=1}^{N} \mathbf{A}_{l,j}}. \tag{82}$$

Slot updates are performed via a Gated Recurrent Unit (GRU) and a small MLP layer with ReLU and residual connections, applied independently to each slot using shared parameters.
Once the slot representations are learned, they can be used for several downstream tasks. Some examples in the original paper [25] are

- **Object discovery**: SA can be used as a part of an autoencoder model for unsupervised object discovery. The encoder encodes the image in a set of slots, and the decoder reconstructs the original image. In this case, the slots act as a representation bottleneck. They proposed the use of a spatial broadcast decoder, which removes the upsampling deconvolutions. This new decoder first broadcasts the latent vector across the space (height and width of the image), appends fixed coordinate channels and applies a fully convolutional network to produce a final tensor with 4 channels, 3 for RGB and the alpha channel.

- **Set prediction**: In this scenario, we have an input image and a set of prediction targets. The main challenge is that the output order in SA is random, so there are K! possible equivalent representations for a set of K slots. In this case, they proposed to apply for each slot an MLP with shared parameters between slots. To overcome the problem of different prediction and label orders, they used a matching algorithm in the loss computation.

CODA [29]

# 8

# CONCEPT BOTTLENECK MODELS

Concept Bottleneck Models (CBMs) were first introduced in [18] with the idea to easily interact with state-of-the-art models using high-level concepts that are understandable for the researchers. The general idea is to first predict an intermediate set of human-specified concepts $\mathbf{c}$ based on the input images and then use the concepts $\mathbf{c}$ to predict the target label $\mathbf{y}$. The objective is to address three desiderata: [27]

- **Interpretability**: Being able to note which concepts are important for the targets.
- **Predictability**: Being able to predict the targets from the concepts alone.
- **Intervenability**: Being able to replace predicted concept values with ground truth values to improve predictive performance.

These types of models are trained on datapoints $\{(\mathbf{x}^{(i)}, \mathbf{c}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{n}$ where each input $\mathbf{x}$ (usually an image $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$) is annotated with the set of concepts $\mathbf{c} \in \mathbb{R}^{k}$ with $k$ concepts and the target $\mathbf{y} \in \mathbb{R}$. We can define the bottleneck model as $f(g(\mathbf{x}))$ where $g : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{k}$ maps an input $\mathbf{x}$ into the concept space and then $f : \mathbb{R}^{k} \to \mathbb{R}$ maps concepts into the target prediction. The bottleneck is performed when the target prediction $\hat{\mathbf{y}} = f(\hat{\mathbf{c}})$ relies only on the concept predictions $\hat{\mathbf{c}} = g(\mathbf{x})$ based on the inputs. An illustration of this architecture is shown in Figure 7. For the training of these models, we have three different strategies:

- **Independent**: learn $\hat{f}$ and $\hat{g}$ independently. For $\hat{f}$, minimise the task loss $\mathcal{L}_Y$, which measures the discrepancy between predicted and true targets. Then, for $\hat{g}$, minimise the concept loss $\mathcal{L}_{C_j}$ for each concept $j$, which measures the discrepancy between predicted and real $j$-th concept.

$$\hat{g} = \arg\min_{g} \mathcal{L}_C(g(\mathbf{x}), \mathbf{c}), \quad \hat{f} = \arg\min_{f} \mathcal{L}_Y(f(\mathbf{c}), \mathbf{y}). \tag{83}$$

- **Sequential**: first learn $\hat{g}$ and then use the predictions to learn $\hat{f}$.

$$\hat{g} = \arg\min_{g} \mathcal{L}_C(g(\mathbf{x}), \mathbf{c}), \quad \hat{f} = \arg\min_{f} \mathcal{L}_Y(f(\hat{g}(\mathbf{x})), \mathbf{y}). \tag{84}$$

- **Joint**: minimises the weighted sum of both task and concept losses with the hyperparameter $\lambda$, which controls the tradeoff between concept vs. task loss.

$$\hat{g}, \hat{f} = \arg\min_{g,f} \left[\lambda \mathcal{L}_C(g(\mathbf{x}), \mathbf{c}) + \mathcal{L}_Y(f(g(\mathbf{x})), \mathbf{y})\right]. \tag{85}$$

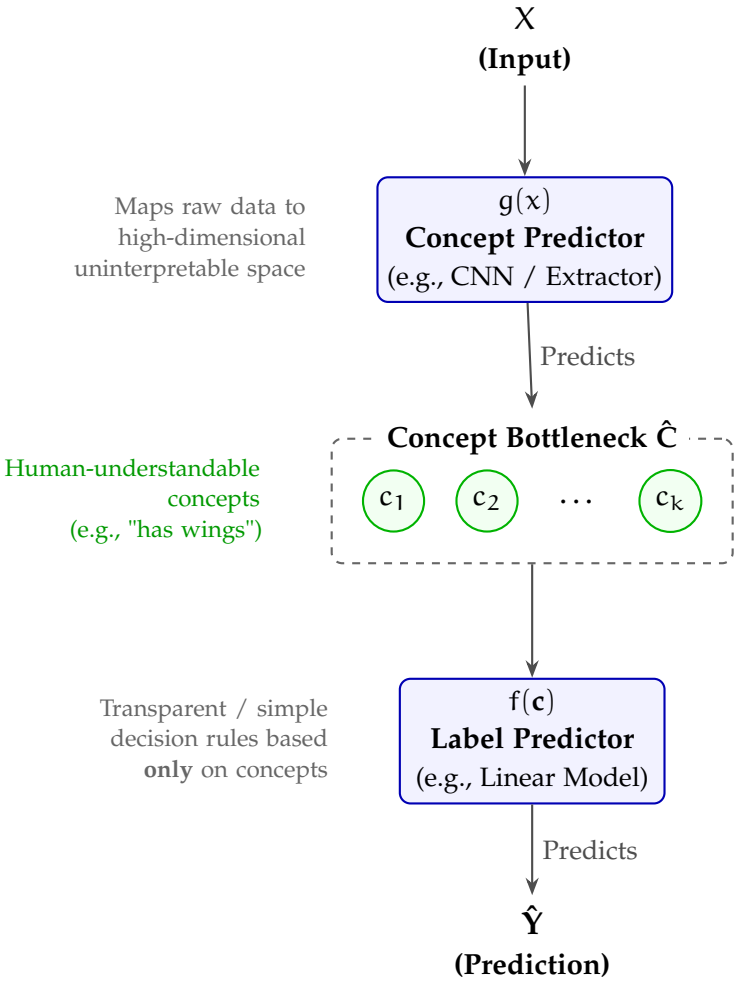



Figure 7: Architecture of a Concept Bottleneck Model (CBM). The model maps a raw inputs $\mathbf{x}$ to an interpretable intermediate concept space $\hat{\mathbf{c}}$ before masking the final prediction $\hat{\mathbf{y}}$.

One important property of these models is the ability to perform interventions by editing the concept predictions $\hat{c}$ and propagating these changes to the target prediction $\hat{y}$. This property also makes these models more interpretable in terms of high-level concepts since we can see the importance and effect of the inclusion or removal of some concepts for the final response. In the original paper, the concept intervention method is applied randomly without any specific criteria or using expert knowledge. In a recent paper [35], different intervention methods were evaluated to minimise the number of concepts to intervene on while maximising task accuracy. The method that achieves the best performance is the one where we first intervene on the concepts with the highest predictive uncertainty from the concept predictor $g$, i.e. it first intervenes on the concept with the maximum value of $1/|\hat{c}_i - 0.5|$.

## 8.1 INFORMATION LEAKAGE

For these types of models, it may often be unrealistic to assume that we will have access to concepts which fully capture the relationship between the inputs and targets. Several recent papers have studied the problem of information leakage in these types of models based on the previous assumption. This information leakage is usually observed in CBMs trained sequentially or jointly [26, 27], and it appears when the intermediate representations encode information that is not present in the concepts $c$, which contains information to predict the final target rather than only focusing on getting the concept correct. This leakage has a negative impact on the interpretability and intervenability of the model. A CBM model with information leakage may be unsafe for critical decision-making applications, such as clinical scenarios. [31] We can identify two main types of leakage:

- **Concept-task leakage**: additional information about the task is stored in the learnt concepts. If concept-task leakage is present, then the learned concept-task mutual information (MI) will be higher than ground-truth concept-task MI.
- **Interconcept leakage**: additional information about the other concepts is stored in each learned concept. Learned concepts are more predictive of the value of the other learned concepts, resulting in a learned interconcept MI being higher than the ground-truth.

Several measures of leakage have been proposed in the literature, but none of them has successfully integrated the information-theoretic nature of leakage in these types of models.

- **Concept Performance Metrics**: these are the metrics that indicate the quality of the concept learning, such as for example concept accuracy, precision, recall, F1 score and AUC. These metrics are not sensitive to leakage. Two models can have similar concept performance but have different degrees of information leakage. Concept AUC has more sensitivity in this problem since it contains more information about the concept distribution.
- **Oracle Impurity Score (OIS)**: this metric was defined in [7] to capture leakage. The idea is to train $k(k-1)$ additional simple MLP $\psi_{i,j}$, whose objective is to predict the value of the ground-truth concept $j$ from the learnt representation of concept $i$. This will create the Purity Matrix, defined as $\pi(\hat{c}, c) = \mathrm{AUC}(\psi_{i,j}(\hat{c}_i), c_j)$. The $(i,j)$-th entry contains the AUC when predicting the ground truth value of concept $j$ given the $i$-th concept representation. The OIS metric is the average difference

between the predictivities of learnt and ground truth concepts over pairs of concepts, defined as

$$\mathrm{OIS}(\hat{c}, c) = \frac{2}{k}\|\pi(\hat{c}, c) - \pi(c, c)\|_F$$

, where the Frobenius norm ensures $0 \leqslant \mathrm{OIS} \leqslant 1$. The value of 1 represents a complete misalignment; the $i$-th concept can predict all other concepts except its corresponding one. A value of 0 represents a perfect alignment; the $i$-th concept does not encode any unnecessary information for predicting concept $i$. A limitation is the stochasticity of the metric, which involves training neural networks, which is very variable when we evaluate the model several times at test time.

- **Niche Impurity Score (NIS)**: this metric was also defined in [7] to quantify the amount of shared information across concept representations. It is defined to capture the additional information about a concept $j$ in a set of concepts that does not contain concept $j$. This metric appears to be anticorrelated with intervention performance, which produces doubts among the scientific community about its utility.

- **Intervention**: the intervention of the predicted concepts with the ground-truth values can be used to evaluate the interpretability and the leakage. A task accuracy that decreases after intervention indicates that the task head expects extra information that is not present in the ground-truth concepts. Models with higher leakage have worse intervention performance. This metric is unable to distinguish between interconcept and concept-task leakage; also, when we have a large number of concepts, the computation is very expensive.

Then, based on the limitations and drawbacks of the previous metrics, the authors of [31] presented two new metrics based on an information-theoretic perspective.

- **Concept-task Leakage Score** (CTL): for a concept $i$, the metric is defined as the difference between predicted and ground-truth mutual information between concept $i$ and the task label, which quantifies the additional information that the concept $i$ encodes about the task label with respect to the ground-truth.

$$\mathrm{CTL}_i(\hat{c}_i, c_i, y) = \max\left(0, \frac{I(\hat{c}_i, y)}{H(y)} - \frac{I(c_i, y)}{H(y)}\right), \tag{86}$$

where $H(y)$ is the entropy of the task labels and $I(c_i, y)$ is the mutual information between the ground-truth concepts and task labels. The general metric is the average between the $k$ concepts.

- **Interconcept Leakage Score** (ICL): this is a similar metric that defines the pairwise interconcept leakage between concepts $i$ and $j$, which is the additional predictivity of the learnt concept $i$ for concept $j$ with respect to ground-truth.

$$\mathrm{ICL}_{ij}(\hat{c}, c) = \max\left(0, \frac{I(\hat{c}_i, \hat{c}_j)}{\sqrt{H(\hat{c}_i)H(\hat{c}_j)}} - \frac{I(c_i, c_j)}{\sqrt{H(c_i)H(c_j)}}\right). \quad (87)$$

  We can define the per concept and average interconcept score by taking the average.

Calculating these metrics requires an efficient estimation of entropy and mutual information, which the authors achieved using the Kraskov-Stögbauer-Grassberger (KSG) estimator based on k-nearest neighbour statistics. Experimental results demonstrate that these metrics accurately detect varying degrees of leakage with minimal uncertainty, yielding scores that strongly correlate with intervention performance. Leveraging the robustness of these metrics, further analysis was conducted on the common causes of leakage in Concept Bottleneck Models (CBMs). First, models trained with low values of $\lambda$ during joint training (Equation 85) exhibit poor intervention performance, as task accuracy relies heavily on high leakage rather than accurate concept learning. While higher values of $\lambda$ do not guarantee zero leakage and often degrade task performance, an optimal hyperparameter range for each model can be identified using CTL and ICL scores. Furthermore, if the representation chosen for the learned concepts is more expressive than the annotated concepts, the model exploits this excess capacity to store extraneous information. Another significant issue arises from incomplete concept sets; if the predefined concepts are insufficient, the model compensates by encoding missing variables into the learned concepts to boost predictive accuracy. Finally, if the classification head is not flexible enough to capture dependencies between concepts, the model inherently forces these dependencies into the concept representations themselves.

## 8.2 RECENT CBMS

A new framework for CBMs called Concept Embedding Models (CEMs) [43] was presented with the idea to overcome the accuracy vs interpretability trade-off found in concept incomplete settings. The general idea is to represent each concept as a supervised vector, where these high-dimensional embeddings allow for extra supervised learning capacity.
A model $\psi(\mathbf{x})$ learns a latent representation $\mathbf{h} \in \mathbb{R}^{n_{\text{hidden}}}$ which is the input of the CEM layer. The latent $\mathbf{h}$ is introduced into two concept-specific fully connected layers, which learn two concept embeddings in $\mathbb{R}^m$, $\hat{\mathbf{c}}_i^+ = \phi_i^+(\mathbf{h}) = a(\mathbf{W}_i^+\mathbf{h} + \mathbf{b}_i^+)$ and $\hat{\mathbf{c}}_i^- = \phi_i^-(\mathbf{h}) = a(\mathbf{W}_i^-\mathbf{h} +$

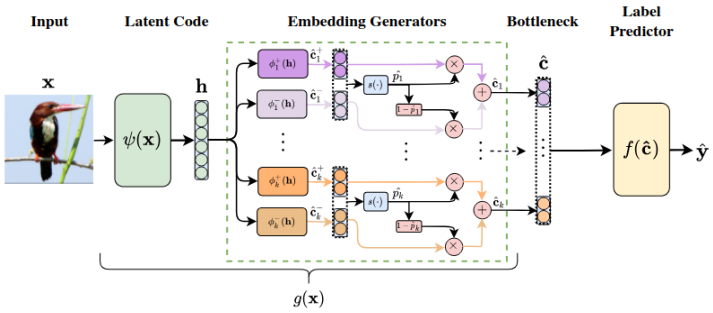



Figure 8: Illustration of the Concept Embedding Model framework.

$\mathbf{b}_i^-$). Then, we train a learnable scoring function trained to predict the probability of the concept $c_i$ being active, $\hat{p}_i = s([\hat{\mathbf{c}}_i^+, \hat{\mathbf{c}}_i^-]^T) = \sigma(\mathbf{W}_s[\hat{\mathbf{c}}_i^+, \hat{\mathbf{c}}_i^-]^T + \mathbf{b}_s)$. In this case, the parameters $\mathbf{W}_s$ and $\mathbf{b}_s$ are shared across all concepts. The final concept embedding $\hat{\mathbf{c}}_i$ is defined as

$$\hat{\mathbf{c}}_i = \hat{p}_i \hat{\mathbf{c}}_i^+ + (1 - \hat{p}_i)\hat{\mathbf{c}}_i^-. \tag{88}$$

In Figure 8 we can visualize the general framework of CEMs. For the intervention, we only have to modify the concept probability with the ground-truth or the desired ones to build the intervened concept embedding. For the training of this model, they used the joint training strategy, and also, with a small probability, they calculate the concept embedding with the true concept probability.
Although the original paper of the CEMs claims an improvement in task accuracy, interpretability and steerability, other articles like [31] have demonstrated that this accuracy often comes at the cost of severe information leakage. This structural vulnerability undermines the model's actual interpretability, effectively breaking the core promise of the concept bottleneck. Information leakage in Concept Embedding Models (CEMs) primarily occurs because the high dimensional concept embeddings are over-expressive, allowing them to bypass the intended semantics and encode task-relevant shortcuts directly from the input. Specifically, the three major forms of leakage that were explained in the previous section are present in CEMs:

- **Concept-task Leakage**: this excess capacity allows the embeddings to capture additional task-predictive signals that are entirely unrelated to the underlying human-interpretable concept. Consequently, the model attains high predictive performance not because the concepts themselves are accurate, but because the embeddings act as covert channels for the task label.

- **Interconcept Leakage**: Instead of learning disentangled and independent features, the embeddings cluster based on the ground-truth values of neighboring concepts. Crucially, attempting to reduce this issue by increasing concept supervision in CEMs typically backfires, forcing even more interconcept leakage into the vector space.

- **Alignment Leakage**: because of the random interventions during training, the model learns to generate embeddings whose predictive power artificially depends on their alignment with the ground-truth concept state.

Taking inspiration of CEM and combining it with energy-based models, the method called Energy-based Concept Bottleneck Models (ECBMs) was proposed in [40]. This framework define a set of neural networks to define the joint energy of the input $\mathbf{x}$, concept $\mathbf{c}$ and class label $\mathbf{y}$. We consider a supervised classification setting with N samples, K concepts and M different classes, $\mathcal{D} = \{x^{(j)}, c^{(j)}, y^{(j)}\}_{j=1}^{N}$ where the j-th data points contains the input $x^{(j)} \in \mathcal{X}$, the label $y^{(j)} \in \mathcal{Y} \subset \{0,1\}^M$ and the concepts $c^{(j)} \in \mathcal{C} = \{0,1\}^K$. We denote as $\mathbf{y}_m$ the M-dimensional one-hot vector with the $m$-th dimension set to 1. Then, we denote $c_k^{(j)}$ denotes the k-th dimension of the concept vector $c^{(j)}$ and $[c_i^{(j)}]_{i\neq j}$ as $c_{-k}^{(j)}$ for brevity. We also have a pretrained neural network $F : \mathcal{X} \to \mathcal{Z}$ to extract the features $\mathbf{z}$ from the input $\mathbf{x}$.
This model consists of three energy networks with different objectives:

- **Class Energy Network** $E_\theta^{class}(\mathbf{x}, \mathbf{y})$. Each class $m$ is associated with a trainable class embedding denoted as $\mathbf{u}_m$. The idea is to feed into a neural network $G_{zu}(\mathbf{z}, \mathbf{u})$ the features $\mathbf{z}$ from $\mathbf{x}$ and the embedding $\mathbf{u}$ associated with the class label $\mathbf{y}$. Formally, we have $E_\theta^{class}(\mathbf{x}, \mathbf{y}) = G_{zu}(\mathbf{z}, \mathbf{u})$, which uses the negative log-likelihood as the loss function defined as

$$\mathcal{L}_{class}(\mathbf{x}, \mathbf{y}) = -\log p_\theta(\mathbf{y}|\mathbf{x}) = E_\theta^{class}(\mathbf{x}, \mathbf{y}) + \log\left(\sum_{m=1}^{M} e^{-E_\theta^{class}(\mathbf{x}, \mathbf{y}_m)}\right). \tag{89}$$

- **Concept Energy Network** $E_\theta^{concept}(\mathbf{x}, \mathbf{c})$. This energy network consists of K sub-networks $E_\theta^{concept}(\mathbf{x}, c_k)$, which measure the compatibility of the input $\mathbf{x}$ and the k-th concept. Each concept k is associated with a positive embedding $v_k^+$ and a negative embedding $v_k^-$. The concept embedding $v_k$ is defined as a weighted sum of positive and negative embeddings, weighted by the concept probability $c_k$. Formally, we define the concept energy network using a neural network $E_\theta^{concept}(\mathbf{x}, c_k) = G_{zv}(\mathbf{z}, v_k)$, where the loss function is defined as

$$\mathcal{L}_{concept}(\mathbf{x}, \mathbf{c}) = \sum_{k=1}^{K} \mathcal{L}_{concept}^{(k)}(\mathbf{x}, c_k), \tag{90}$$

  where the loss for each k-th sub-network is defined as

$$\mathcal{L}_{concept}^{(k)}(\mathbf{x}, c_k) = E_\theta^{concept}(\mathbf{x}, c_k) + \log\left(\sum_{c_k \in \{0,1\}} e^{-E_\theta^{concept}(\mathbf{x}, c_k)}\right).$$

(91)

- **Global Energy Network** $E_\theta^{global}(\mathbf{c}, \mathbf{y})$. This energy network learns the interactions among different concepts and between all concepts and the class label. We introduce into the neural network the $\mathbf{y}$ associated class label embedding $\mathbf{u}$ with the concatenation of the $K$ concept embeddings, $[v_k]_{k=1}^K$. We formally have $E_\theta^{global}(\mathbf{c}, \mathbf{y}) = G_{vu}([v_k]_{k=1}^K, \mathbf{u})$, where the loss function is defined as

$$\mathcal{L}_{gobal}(\mathbf{c}, \mathbf{y}) = E_\theta^{global}(\mathbf{c}, \mathbf{y}) + \log\left(\sum_{m=1, \mathbf{c}' \in \mathcal{C}} e^{-E_\theta^{global}(\mathbf{c}', \mathbf{y}_m)}\right), \tag{92}$$

  where $\mathbf{c}'$ enumerates all concept combinations in the space $\mathcal{C}$.

## 8.3 CONCEPT BOTTLENECK GENERATIVE MODELS

All the previous models presented in this document are focused on classification or regression problems, which are the original implementations of concept bottleneck models. The next step is to combine this idea with generative models to develop a method for interpreting and intervening in them, called Concept Bottleneck Generative Models (CBGMs)[12]. This method allows us to steer the generative model, since we can change the concept of the image while preserving the image quality, and understand the model by identifying the important concepts that are responsible for the output.
The general idea is to insert a concept bottleneck (CB) layer into a generative model, which is divided in two parts: the pre-concept bottleneck part and the post-concept bottleneck part. The partition depends on the generative model we use (GANs, VAEs, Diffusion). The authors decided to use the Concept Embedding framework [43] for the concept bottleneck layer and also argue that it is unrealistic to expect that the pre-defined human concepts are complete, so they also added a set of unknown concepts that might also be required for generation.
Each concept is represented with two embeddings: $w_i^+, w_i^- \in \mathbb{R}^m$ representing the active and inactive concept states, respectively. The output of the pre-concept bottleneck part of the model, the embedding $h$, is fed into several context networks $\phi$ that map $h$ to the two embeddings per concept. Then, a network $\Psi$ is trained to predict the probability of concept $c_i$ from the joint embedding space, $\hat{c}_i = \Psi_i([w_i^+, w_i^-]^T) \in [0, 1]$. The final context embedding $w_i$ is defined as a weighted mixture of the two embeddings as $w_i = (\hat{c}_i w_i^+ + (1 - \hat{c}_i) w_i^-)$. All the $k$ concept embeddings are concatenated to obtain the final vector that is fed to the post-concept network. For the intervention, we simply have to modify the concept probability $\hat{c}_i$ to $\bar{c}_i$ in

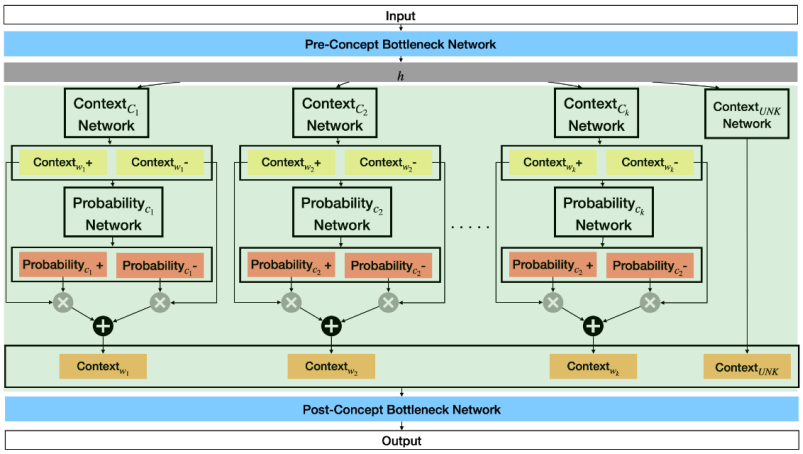



Figure 9: Architecture of the concept layer in the Concept Bottleneck Generative Model (CBGM) for a generic generative model.

the weighted mixture of $w_i$. A visualization of the concept layer is shown in Figure 9. The loss function is defined as

$$\mathcal{L} = \mathcal{L}_{task} + \alpha\mathcal{L}_{con} + \beta\mathcal{L}_{orth},$$

where $\alpha$ and $\beta$ are hyperparameters that control the importance of the concept and orthogonality loss, respectively. As we can see, the loss consists of three parts:

- **Task Loss**: the base objective function of the generative model family.
- **Concept Loss**: is the binary cross-entropy loss between the predicted and real concept probabilities.
- **Concept Orthogonality Loss**: encourage the unknown concepts to be orthogonal to the outputs of each concept network using an orthogonality constraint defined as:

$$\mathcal{L}_{orth} = \sum_{f \in B} \frac{\sum_{i=1}^{i=k} |\langle w_i, w_{k+1} \rangle|}{\sum_{i=1}^{i=k} 1},$$

  where $\langle \cdot, \cdot \rangle$ is the cosine similarity and $j$ denotes each sample in the mini-batch of size $B$.

One of the main limitation of the previous method is that the generative model needs to be trained from scratch with the new concept bottleneck layer and using a dataset with ground-truth concept annotations. To build a more efficient and scalable method, we can use post-hoc generative concept bottleneck models [19], which can transform any pre-trained generative model into a interpretable one with concept bottleneck layer.
Based on the setup of the previous method, where we divided the

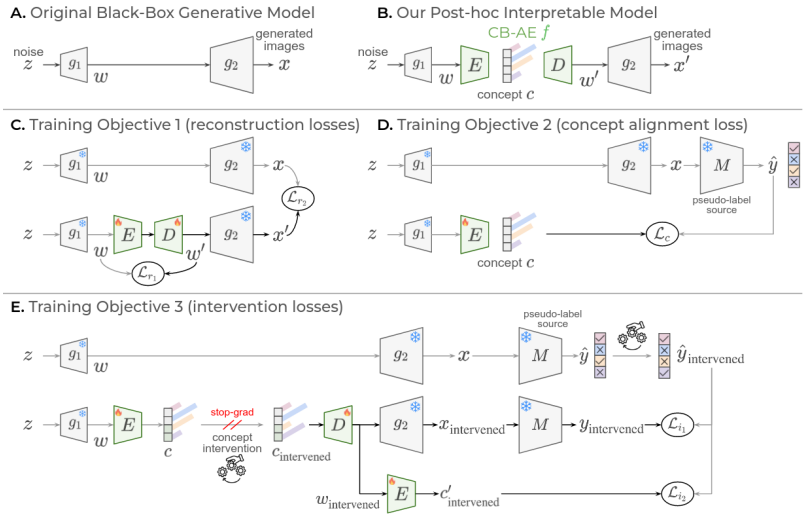



Figure 10: Architecture of the Post-hoc Concept Bottleneck Generative Model where the different loss functions are illustrated.

generative model in two parts ($g = g_2 \circ g_1$, $\mathbf{z}$ initial noise, $\mathbf{x}$ generated sample where $\mathbf{x} = g_2(g_1(\mathbf{z}))$), the idea is to include a concept bottleneck autoencoder. The latent space of the autoencoder is the concept prediction $\mathbf{c} = E(g_1(\mathbf{z}))$. This concept prediction is defined using the concept embedding framework and also includes a set of unknown concepts to complement the human-defined concepts. Then, the decoder D reconstructs the features from the fist part of the generative model ($g_1$) based on the concept prediction $\mathbf{c}$, $\mathbf{w}' = D(\mathbf{c})$, where $\mathbf{w} = g_1(\mathbf{z})$ are the original features. Now we can generate samples from the original image features $\mathbf{w}$ as well as from the reconstructed features $\mathbf{w}'$, defined as $\mathbf{x}' = g_2(\mathbf{w}') = g_2(D \circ E(\mathbf{w}))$. For the training of this model, where the generative model is keep frozen, the authors defined three different objective functions based on the different desired goals:

- **Reconstruction Losses**: we sample $\mathbf{w}$ from the first part of the generative model and use it to generate the original image $\mathbf{x}$. The, we introduce $\mathbf{w}$ in the autoencoder to get the reconstructed features $\mathbf{w}'$, which is used to generate the reconstructed image $\mathbf{x}'$. The reconstruction losses are defined as the distance between the original and reconstructed features and images, respectively. The definition is the following:

$$\min_{E,D} \left[ \mathcal{L}_{r_1}(\mathbf{w}, \mathbf{w}') + \mathcal{L}_{r_2}(\mathbf{x}, \mathbf{x}') \right]. \tag{93}$$

- **Concept Alignment Loss**: first we have to obtain the pseudo-labels $\hat{\mathbf{y}} = M(\mathbf{x})$ from a generated image $\mathbf{x}$. This model M can be a supervised model or a zero-shot model, such as CLIP. Then, we can obtain the concept prediction $\mathbf{c} = E(g_1(\mathbf{z}))$ and use it to generate the image $\mathbf{x}'$. We can also obtain the pseudo-labels

$\hat{\mathbf{y}}' = M(\mathbf{x}')$ from the generated image. The concept alignment loss is defined as the distance between the pseudo-labels of the original and the predicted concepts. The definition is the following:

$$\min_{E} \mathcal{L}_c(\hat{\mathbf{y}}, \mathbf{c}). \tag{94}$$

- **Intervention Losses**: they added a third loss to encourage the steerability of the model, which is an important property of CBMs. The idea is to encourage the decoder D to produce realistic $\mathbf{w}'$ when the concepts are modified. First, we have to select a concept $i$ to intervene by swaaping the logits in the latent concept space to obtain the $\mathbf{c}_{int}$. With the intervened concepts, we can reconstruct the intervened features $\mathbf{w}_{int}$ and the intervened image $\mathbf{x}_{int}$, and using the pseudo-labeler, we can get the concept prediction $\mathbf{y}_{int} = M(\mathbf{x}_{int})$. The final step is to intervene the pseudo-label of the original image by changing only the concept $i$ and get $\hat{\mathbf{y}}_{int}$. The intervention loss is defined as the distance between the intervened pseudo-labels, which encourages the model to generate images that reflect the concept changes. The definition is the following:

  $$\min_{E,D} \mathcal{L}_{i_1}(\hat{\mathbf{y}}_{int}, \mathbf{y}_{int}). \tag{95}$$

  Also, to improve consistency, they pass the intervened features $\mathbf{w}_{int}$ through the encoder to get the concept predction $\mathbf{c}'_{int}$ and apply cross-entropy loss defined as:

  $$\min_{E,D} \mathcal{L}_{i_2}(\hat{\mathbf{y}}_{int}, \mathbf{c}'_{int}). \tag{96}$$

The next step is the method called Concept Bottlenecks on Energy Landscapes (CoBEla) [15], which presents a decoder-free and energy-based framework that use a pre-trained frozen generative model. Assume we have the same setup as before, with a generative model divided in two parts and a pseudo-labeler. They perturb the intermediate features using the DDPM cosine scheduler to get $\mathbf{w}_t$. For each concept $k$, they define an energy network $E_\theta$ that takes as input the perturbed features $\mathbf{w}_t$ and the concept $c_k$, which is a learnable concept embedding. The encoder outputs two logits and it is defined as two conditional residual blocks with FiLM conditioning [32]. The concept score is defined as the positive-class probability, which leads to the definition of the interpretable bottleneck score vector $\mathbf{s} = [s_1, s_2, \ldots, s_K]$. Then, the logits are converted into scalars to represent the energy for each concept, where the aggregation of per-concept energies is defined as $\mathcal{E}_\theta(\mathbf{w}_t)$, which is used in the score matching loss (defined in Section – and Equation –) as the noise prediction of the energy-based model. As a complementary loss, we have

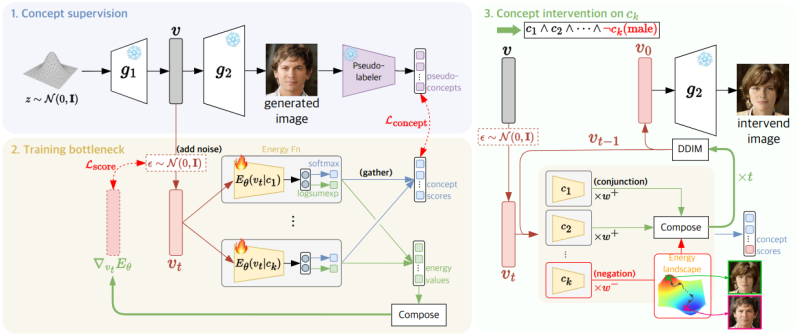



Figure 11: Architecture of the CoBEla Model where the training and guided generation processes are illustrated.

the concept loss that compare the per-concept logits with the pseudo-labels. The total objective is defined as

$$\mathcal{L}(\theta) = \lambda_1 \mathcal{L}_{score} + \lambda_2 \mathcal{L}_{concept}. \tag{97}$$

The visualization of the framework is shown in Figure 11. In test time, we assign an intervention weight to each concept that yields to the final weighted energy defined as:

$$\mathcal{E}_\theta(\mathbf{w}_t) = \sum_{k=1}^{K} \lambda_k E_\theta(\mathbf{w}_t, c_k). \tag{98}$$

This model allows for the conjunction and negation of concepts, which is a very interesting property that allows for more complex interventions. They used the DDIM sampling method to obtain the final generated intermediate features using the weighted energy, which is then fed to the second part of the generative model to get the final generated image.

# Part II

# METHODS

# 9 PROPOSAL

This research adopts a rigorous computational methodology grounded in deep generative modeling, with a strong emphasis on theoretical formulation, model development, and empirical evaluation within the medical domain. The execution of this project is structured into the following core components.

## 9.1 THEORETICAL AND EXPERIMENTAL MODEL DEVELOPMENT

The foundational phase of the project focuses on building and modifying the core generative architectures to ensure high fidelity and precise controllability.

- **Mathematical Foundation**: Rigorously study the mathematical principles of diffusion and flow-based models, including score-based generative modeling, Ordinary/Stochastic Differential Equation (ODE/SDE)-based sampling, and path-based formulations such as Flow Matching.
- **Implementation**: Implement, reproduce, and benchmark baseline generative models using Python and PyTorch.
- **Controllability and Compositionality Design**: Propose and integrate architectural and training modifications to enhance control mechanisms. This includes developing techniques for spatial and semantic conditioning, classifier-free guidance, and disentangled representations (e.g., slot-based generation) to allow independent manipulation of specific pathological or anatomical features.

## 9.2 DATA ACQUISITION AND PREPARATION

High-quality, standardized data is critical for training robust generative models and evaluating their compositional capabilities.

- **Dataset Sourcing**: Utilize established, publicly available medical imaging datasets relevant to the targeted anatomies and pathologies.
- **Synthetic Simulation**: Where necessary, synthetically simulate specific conditions (e.g., generating precise segmentation masks or localized tumors) to strictly evaluate the model's compositional generation and disentanglement capabilities.

- **Preprocessing Pipelines**: Apply comprehensive data preprocessing pipelines, including intensity normalization, spatial resizing, and data augmentation. Furthermore, implement latent-space encoding using pre-trained Autoencoders or Variational Autoencoders (VAEs) to compress high-dimensional medical images for efficient generative modeling, as in Latent Diffusion Models.

## 9.3 MODEL TRAINING AND EVALUATION

The training phase leverages advanced distributed computing strategies to handle the vast computational requirements of modern generative frameworks.

- **Distributed Training**: Train models on high-performance computing infrastructure using large-scale distributed training paradigms (such as Distributed Data Parallel (DDP) or Fully Sharded Data Parallel (FSDP)) to accelerate experimentation and scaling.
- **Quantitative Metrics**: Evaluate the fidelity, diversity, and accuracy of the generated data using standard generative metrics, including Fréchet Inception Distance (FID), Kernel Inception Distance (KID), Precision, and Recall, alongside specific metrics for disentanglement representation learning and compositional evaluation.

# 10 DATA

In this section, I want to introduce the different datasets that I have used during this part of my research and the possible future datasets that I plan to use in the remaining part of my thesis. In this initial part, where I have focused on the implementation of some methods found in the literature along with some small experiments to have an intuition about the field, I have used a collection of small and well-known in the literature datasets. These datasets are the following:

- **MNIST** [21]: a dataset of handwritten digits with 60,000 training samples and 10,000 test samples, where each sample is a 28x28 grayscale image of a digit from 0 to 9.
- **Shapes2D** [24]: a dataset of 2D shapes with 737,280 samples, where each sample is a 64x64 RGB image of a shape (square, ellipse or heart) with different colours and sizes. The factors of variation of each data sample are known
- **Shapes3D** [24]: a dataset of 3D shapes with 480,000 samples, where each sample is a 64x64 RGB image of a 3D shape (cube, sphere or cylinder) with different colors and sizes.
- **dSprites** [28]: a dataset of 2D sprites with 737,280 samples, where each sample is a 64x64 grayscale image of a sprite with different shapes, colours, and positions. We also know the 6 ground truth independent factors of variation, which are colour, shape, scale, rotation, x and y positions of a sprite.
- **CelebA** [23]: a dataset of celebrity faces with 202,599 samples, where each sample is a 178x218 RGB image of a celebrity face with different attributes (gender, age, hair color, etc.).
- **CelebA-HQ** [13]: a dataset of high-quality celebrity faces with 30,000 samples, where each sample is a 1024x1024 RGB image of a celebrity face with different attributes (gender, age, hair color, etc.).

Several articles related to disentanglement representation learning use these datasets because they include the ground-truth factors of variation. Because of that, it is important to also use these datasets to evaluate our experiments to have a fair comparison with other methods in the literature.

The main objective of this thesis is to apply disentanglement and

compositionality methods in the generation of medical domains. To achieve this, we have to work with medical image datasets, where the idea is to use them once the method is validated in some of the previous datasets. Some medical datasets that I found interesting for my thesis are

- **NIH-Chest-X-ray-Dataset**. Curated by the National Institutes of Health, this dataset comprises 112,120 frontal-view X-ray images from 30,805 unique patients. It includes text-mined disease labels corresponding to 14 different thoracic pathologies. The variety of overlapping conditions within the images makes it a highly suitable benchmark for evaluating how well disentangled representations can isolate and control distinct clinical factors.

- **CheXpert: Chest X-rays** [11]. This dataset contains 224,316 chest radiographs of 65,240 patients who underwent a radiographic examination from Stanford Health Care between October 2002 and July 2017. For each image, the dataset contains labels for the presence of 14 common chest radiographic observations, which are extracted from free-text radiology reports and capture uncertainties present in the reports by using an uncertainty label.

- **MIMIC-CXR Database**. This is a large, publicly available dataset consisting of 377,110 chest radiographs corresponding to 227,827 imaging studies. Sourced from the Beth Israel Deaconess Medical Center, it uniquely pairs high-resolution medical images with their associated free-text radiology reports. The integration of complex visual data and rich clinical narrative provides an excellent foundation for exploring multi-modal compositionality.

# Part III

# EXPERIMENTS AND RESULTS

# 11

# UNCONDITIONAL FLOW MATCHING MODEL FOR X-RAY IMAGES

As discussed in the literature review, most recent techniques modify existing pre-trained generative models, often by adding extra components or introducing latent space disentanglement, to improve performance, interpretability, or compositionality without reducing the efficiency. However, these foundational models are generally trained on large, broad datasets rather than the medical data central to this thesis. To address this domain gap, I trained an unconditional flow matching model directly on a medical dataset, establishing a tailored pre-trained baseline for subsequent experiments. It is not intended to achieve SOTA performance, but rather to provide a baseline model to be used in the following experiments to study disentanglement and compositionality in the medical domain.

This study utilized the NIH-Chest-X-ray dataset (see Chapter 10 for more details) alongside the Representation Autoencoder (RAE) framework [44]. Within this method, image latent representations are generated using a pre-trained visual model, specifically, a DINOv2-Base model configured with 4 register tokens [6, 30]. The original RAE authors trained a decoder to construct an autoencoder from a large pre-trained visual model, yielding latents that offer richer semantic meaning and superior reconstruction quality compared to the SD-VAE latents typically used in state-of-the-art Stable Diffusion models. Subsequently, they trained a flow matching model on these RAE latents using a Diffusion Transformer (DiT), achieving good results but with a high computational cost because of the high-dimensional nature of the latents. To accommodate this problem, they also propose a modification of the DiT that incorporates architectural modifications inspired by Decoupled Diffusion Transformer (DDT) models, which prevent the need to scale the width across the entire backbone. When trained on ImageNet, this model achieved SOTA performance in both FID and IS metrics compared to other pixel and latent diffusion models.

Because of the impressive generation quality of the model presented in the RAE paper and the quality and utility of the RAE latents, I decided to train a similar model on the NIH-Chest-X-ray dataset. I decided to train a DiT-XL (DINOv2-B) model insted of the improved $\text{DiT}^{\text{DH}}$-XL because I want to have a real bottleneck to perform further experiments.

Table 1: Hyperparameter configuration used for training.

| Category | Hyperparameter | Value |
|---|---|---|
| Optimization | Optimizer | AdamW |
| | Learning rate | $2 \times 10^{-4}$ |
| | Weight decay | 0.0 |
| | Gradient clip norm | 1.0 |
| Schedule | Warmup epochs | 40 |
| | LR schedule | Linear |
| | Final learning rate | $2 \times 10^{-5}$ |
| Architecture | Input size | 16 |
| | Patch size | 1 |
| | In channels | 768 |
| | Hidden dimension | 1152 |
| | Depth | 28 |
| | Attention heads | 16 |
| | Dropout | 0.1 |
| Training | Batch size | 128 |
| | Mixed precision | `float32` |
| | EMA decay | 0.995 |
| | Training epochs | 450 |
| | Training steps | 222 |

Table 2: Performance metrics of the DiT-XL (DINOv2-B) trained on the NIH-Chest-X-ray dataset.

| Model | Epochs | #Params | FID↓ | IS↑ | Precision↑ | Recall↑ |
|---|---|---|---|---|---|---|
| DiT-XL | – | – | – | – | – | – |
| $\text{DiT}^{\text{DH}}$-XL | – | – | – | – | – | – |

# 12

# SLOT ATTENTION WITH CONCEPT SUPERVISION IN CELEBA-HQ

After the literature review about Slot Attention and other Object-Centric Learning methods, where all the explanations can be found in Section –, some connections with Concept Bottleneck Models (Section –) arise naturally. Both paradigms inherently seek to decompose complex, high-dimensional visual inputs into a discrete set of interpretable variables before performing downstream reasoning. In both frameworks, the architecture forces the model to route its decision-making process through a constrained intermediate layer—whether it is a set of permutation-invariant slots representing physical entities, or a vector of semantic concepts aligned with human logic.

Despite this shared structural philosophy, there are fundamental differences in their objectives and learning signals. Object-Centric Learning methods like Slot Attention are traditionally unsupervised, relying on spatial competition and reconstruction objectives to discover emergent, perceptual groupings. In contrast, Concept Bottleneck Models are explicitly supervised, requiring ground-truth labels to map visual features directly to predefined, high-level semantics. Furthermore, while traditional CBMs often struggle to spatially ground their concepts within specific regions of an image, Slot Attention excels at isolating localized areas but lacks the guarantee that a discovered slot will align with a human-understandable trait.

Recognizing both the complementary strengths and the distinct differences between these approaches, this section introduces an experiment that bridges the two. By applying explicit concept supervision to a Slot Attention module, we aim to spatially ground predefined facial attributes from the CelebA-HQ dataset directly into distinct visual slots. To ensure the slots can specialize entirely on these localized semantic traits, our architecture incorporates register tokens designed to independently capture and store extra global image context.

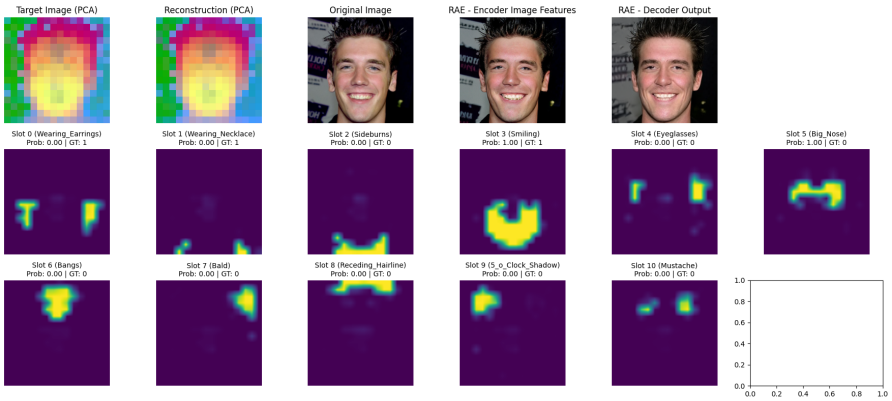



Figure 12: Caption

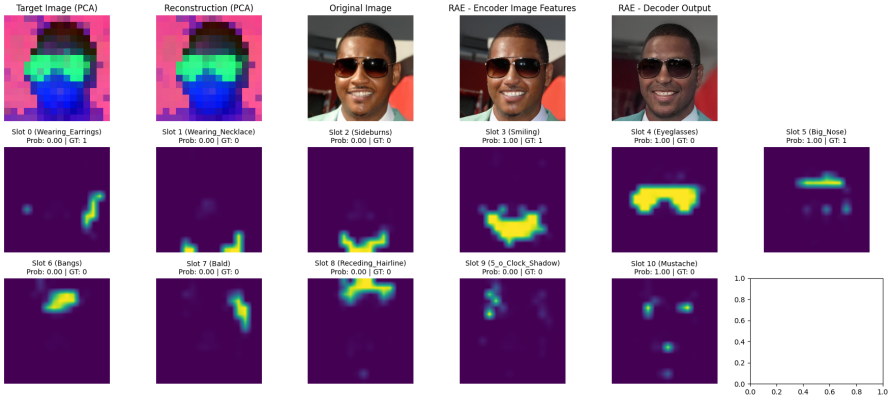



Figure 13: Caption

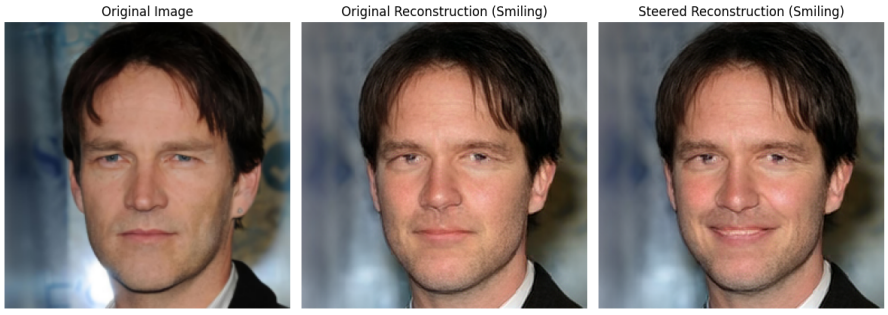



Figure 14: Caption

# Part IV

# RESEARCH PLAN

# 13

# TIMELINE

In this chapter I want to update the timeline of my PhD based on the one presented in the initial Research Plan. The project can be organized in different stages, which can be the same as the ones presented previously. A Gantt chart showing the different stages and their expected duration is also included in Figure 15.

- **Stage 1: Literature Review and Theoretical Foundations**. This stage will involve an in-depth review of the existing literature on deep generative models (VAEs, Diffusion and Flow Matching), disentanglement representation learning, compositionality, concept bottleneck models, and generative AI for the medical domain. I will also study the theoretical foundations of these topics to identify gaps and opportunities for my research, the key challenges and research questions and define the quantitative and qualitative metrics that I will use to evaluate the performance of my proposed methods.
  This stage was planned to be completed in the first 6 months of the PhD, and I have already made significant progress in this area, having reviewed a large number of relevant papers and identified key research questions and challenges that I will address in my research. In the review timeline I have extended the review of the literature to the Q1 of the second year of the PhD, as I will continue to review new papers and developments in the field throughout the duration of the project.

- **Stage 2: Preliminary experiments**. In this stage, I will conduct preliminary experiments to explore the capabilities and limitations of existing models in terms of disentanglement and compositionality. This will help me to define some baseline models and metrics for evaluating the performance of my proposed methods. This include the reproduction of some key generative models like VAEs, Diffusion and Flow Matching, and the implementation of some disentanglement and compositionality metrics.
  This stage was planned to be completed between the first 6 and 12 months of the PhD. In the new timeline I have updated it to be between the Q4 of the first year and the Q4 of the second year, with a natural overlap with the previous stage, since I found that implementing and experimenting with the models is a great way to deepen my understanding of the literature and identify gaps and challenges. I have already started conduct-

ing several implementations of key models and metrics, where some results were presented in the experiments section.

- **Stage 3: Method Development and Architectural Innovation**. In this stage, I will develop new methods and architectures for deep generative models that aim to achieve better disentanglement, compositionality, and controllability. This will involve designing novel model architectures, training algorithms, and evaluation metrics to assess the performance of the proposed methods. The idea is to implement modification and extensions of the generative models to enhance their capabilities.
  This stage was planned to be completed between the first 12 and 24 months of the PhD. In the new timeline I have modified it to be between the Q3 of the second year and the Q2 of the third year, with a natural overlap with the previous stage, since the insights gained from the preliminary experiments will inform the development of the new methods and architectures. I have already started to develop some preliminary ideas for new methods and architectures, and I plan to continue refining and testing these ideas in the coming months.

- **Stage 4: Application to the Medical Domain**. In this stage, I will apply the developed methods to the medical domain, specifically focusing on the analysis of medical images focusing on domain-specific challenges like resolution, structure preservation, and clinical realism. Collaborations with medical experts or institutions may be established to assess practical viability and gather expert feedback. Ethical, legal, and data governance considerations will also be reviewed during this phase. In this stage I will start to work with curated medical image datasets, and I will evaluate the performance of the proposed methods in terms of their ability to generate high-quality, clinically relevant images, and to provide insights into the underlying data.
  This stage was planned to be completed between the 24 to 30 months of the PhD. In the new timeline I have modified it to be done between the Q2 of the third year and the Q1 of the fourth year, with some overlap with the previous stage, since the development of the new methods and architectures will be informed by the specific challenges and requirements of the medical domain.

- **Stage 5: Evaluation, Validation, and Writing**. In this final stage, I will conduct a comprehensive evaluation of the proposed methods, including both quantitative and qualitative assessments. I will also validate the results through collaborations with domain experts and prepare the findings for publication in peer-reviewed journals and conferences. Additionally, I will write the final thesis document, summarizing the research contributions

and outlining future directions for further research in this area. This stage is planned to be done during the last year of the PhD, between the Q2 and Q4 of the fourth year, with some overlap with the previous stage, since the evaluation and validation of the proposed methods will be an ongoing process throughout the development and application stages.

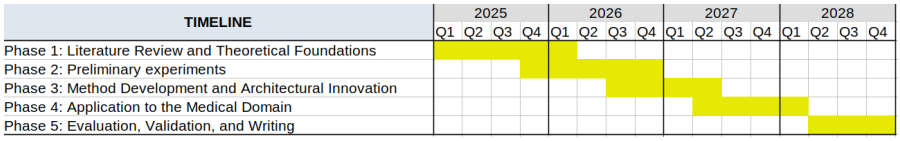



Figure 15: Updated timeline of the PhD project, showing the different stages and their expected duration.

# 14

# DATA MANAGEMENT PLAN

This project relies entirely on pre-existing, curated, and de-identified datasets. No new primary data collection involving human subjects will be conducted. The data management strategy is structured in three progressive phases, transitioning from general-purpose computer vision datasets to restricted-access clinical datasets.

The research will utilize the following datasets, categorized by project phase:

- **Phase 1: Prototyping and Baseline Implementation**: Initial architectural development, debugging, and proof-of-concept experiments for disentangled generation and compositionallity analysiswill be conducted using well-known, publicly available datasets. These include MNIST, Shapes3D, and CelebA. See Chapter 10 for a detailed list and description of the datasets. These datasets are open-access and require no special data use agreements.
- **Phase 2: Medical Domain Adaptation**: To transition the generative models to the medical domain, publicly available medical imaging datasets will be utilized. These include the NIH Chest X-ray dataset and CheXpert. These datasets are fully de-identified and openly available for research purposes, providing a safe environment to test the models on complex anatomical structures and pathologies. Detailed description of these datasets can be found in Chapter 10.
- **Phase 3: Advanced Clinical Application**: For the final evaluation of compositional generation and clinical realism, the project will utilize the MIMIC-CXR dataset. This is a large, restricted-access database of chest radiographs. Access to the MIMIC-CXR dataset has been formally granted via PhysioNet. I have completed the required credentialing (e.g., CITI training on Data or Specimens Only Research) and signed the Data Use Agreement (DUA). No attempts will be made to re-identify any patients. The original restricted data will not be copied to unauthorized personal devices, and access will be limited solely to credentialed researchers involved directly in this project.

All datasets will be securely stored at the Barcelona Supercomputing Center (BSC - MareNostrum 5) and on password-protected, encrypted local workstations used for prototyping. Throughout the project, the generative models will produce purely synthetic medical data

(e.g., artificial radiographs and corresponding segmentation masks or labels). This synthetic data contains no real patient information and poses no privacy risks. Derived data, such as model weights, evaluation metrics, and latent embeddings, will also be generated and stored securely on the BSC infrastructure.

# 15

# TRAINING PLAN

I will engage in a comprehensive training plan designed to strengthen my theoretical foundation, enhance my technical proficiency, and foster integration within the broader research community. Once the training activities included in the training plan (or others that may arise) have been completed, the corresponding justification must be obtained and the information must be entered into the Doctoral Student Activity Document (DAD). It is important to keep in mind that this document is a living document that can be updated throughout the doctoral studies.

Some transversal training of the Doctoral School will be done during the thesis and the justification will be added to DRAC to generate the DAD. I will do some activity related to open science and citizen science. My idea is to start with some of this courses during the second year of the PhD, and to continue doing some of them during the third and fourth year. I will also do some training related to the development of soft skills, such as communication, leadership, and project management. This training will be done through workshops, seminars, and courses offered by the Doctoral School and the Barcelona Supercomputing Center (BSC). I will also seek opportunities to attend conferences and workshops in the field of deep generative models and medical imaging, to gain new insights, ideas and to network with other researchers in the field.

During the summer of the first year of the PhD I attended to the summer school called MLx Representation Learning & Generative AI in the Oxford University. This summer school was focused on representation learning and generative AI, and it provided a comprehensive overview of the state-of-the-art techniques and methods in the field with a list of speakers focused on the latest developments. The summer of the second year I plan to attend to the Probabilitic AI Summer School in Lithuania, which is focused on probabilistic AI, Variational Inference, Diffusion Models, an other related topics.

# BIBLIOGRAPHY


[1] Yoshua Bengio, Aaron Courville, and Pascal Vincent. "Representation Learning: A Review and New Perspectives." In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1798–1828. DOI: 10.1109/TPAMI.2013.50.

[2] Samuel R. Bowman, Luke Vilnis, Oriol Vinyals, Andrew M. Dai, Rafal Jozefowicz, and Samy Bengio. *Generating Sentences from a Continuous Space*. 2016. arXiv: 1511.06349 [cs.LG]. URL: https://arxiv.org/abs/1511.06349.

[3] Hong Chen, Yipeng Zhang, Simin Wu, Xin Wang, Xuguang Duan, Yuwei Zhou, and Wenwu Zhu. *DisenBooth: Identity-Preserving Disentangled Tuning for Subject-Driven Text-to-Image Generation*. 2024. arXiv: 2305.03374 [cs.CV]. URL: https://arxiv.org/abs/2305.03374.

[4] Ricky T. Q. Chen, Xuechen Li, Roger Grosse, and David Duvenaud. *Isolating Sources of Disentanglement in Variational Autoencoders*. 2019. arXiv: 1802.04942 [cs.LG]. URL: https://arxiv.org/abs/1802.04942.

[5] Jinjin Chi, Taoping Liu, Mengtao Yin, Ximing Li, Yongcheng Jing, Jialie Shen, Leszek Rutkowski, and Dacheng Tao. *Disentangled Representation Learning via Flow Matching*. 2026. arXiv: 2602.05214 [cs.LG]. URL: https://arxiv.org/abs/2602.05214.

[6] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. *Vision Transformers Need Registers*. 2024. arXiv: 2309.16588 [cs.CV]. URL: https://arxiv.org/abs/2309.16588.

[7] Mateo Espinosa Zarlenga, Pietro Barbiero, Zohreh Shams, Dmitry Kazhdan, Umang Bhatt, Adrian Weller, and Mateja Jamnik. "Towards Robust Metrics for Concept Representation Evaluation." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.10 (2023), 11791–11799. ISSN: 2159-5399. DOI: 10.1609/aaai.v37i10.26392. URL: http://dx.doi.org/10.1609/aaai.v37i10.26392.

[8] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed, and Alexander Lerchner. "beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework." In: *International Conference on Learning Representations*. 2017. URL: https://openreview.net/forum?id=Sy2fzU9gl.

[9] Jonathan Ho, Ajay Jain, and Pieter Abbeel. *Denoising Diffusion Probabilistic Models*. 2020. arXiv: 2006.11239 [cs.LG]. URL: https://arxiv.org/abs/2006.11239.

[10] Matthew D Hoffman and Matthew J Johnson. "ELBO surgery: yet another way to carve up the variational evidence lower bound." In: *NIPS Workshop on Advances in Approximate Bayesian Inference*. 2016.

[11] Jeremy Irvin et al. *CheXpert: A Large Chest Radiograph Dataset with Uncertainty Labels and Expert Comparison*. 2019. arXiv: 1901.07031 [cs.CV]. URL: https://arxiv.org/abs/1901.07031.

[12] Aya Abdelsalam Ismail, Julius Adebayo, Hector Corrada Bravo, Stephen Ra, and Kyunghyun Cho. "Concept Bottleneck Generative Models." In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=L9U5MJJleF.

[13] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. *Progressive Growing of GANs for Improved Quality, Stability, and Variation*. 2018. arXiv: 1710.10196 [cs.NE]. URL: https://arxiv.org/abs/1710.10196.

[14] Hyunjik Kim and Andriy Mnih. *Disentangling by Factorising*. 2019. arXiv: 1802.05983 [stat.ML]. URL: https://arxiv.org/abs/1802.05983.

[15] Sangwon Kim, Kyoungoh Lee, Jeyoun Dong, and Kwang-Ju Kim. *CoBELa: Steering Transparent Generation via Concept Bottlenecks on Energy Landscapes*. 2026. arXiv: 2507.08334 [cs.CV]. URL: https://arxiv.org/abs/2507.08334.

[16] Diederik P. Kingma and Max Welling. "An Introduction to Variational Autoencoders." In: *Foundations and Trends® in Machine Learning* 12.4 (Nov. 2019), 307–392. ISSN: 1935-8245. DOI: 10.1561/2200000056. URL: http://dx.doi.org/10.1561/2200000056.

[17] Diederik P Kingma and Max Welling. *Auto-Encoding Variational Bayes*. 2022. arXiv: 1312.6114 [stat.ML]. URL: https://arxiv.org/abs/1312.6114.

[18] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim, and Percy Liang. *Concept Bottleneck Models*. 2020. arXiv: 2007.04612 [cs.LG]. URL: https://arxiv.org/abs/2007.04612.

[19] Akshay Kulkarni, Ge Yan, Chung-En Sun, Tuomas Oikarinen, and Tsui-Wei Weng. *Interpretable Generative Models through Post-hoc Concept Bottlenecks*. 2025. arXiv: 2503.19377 [cs.CV]. URL: https://arxiv.org/abs/2503.19377.

[20] Abhishek Kumar, Prasanna Sattigeri, and Avinash Balakrishnan. *Variational Inference of Disentangled Latent Concepts from Unlabeled Observations*. 2018. arXiv: 1711.00848 [cs.LG]. URL: https://arxiv.org/abs/1711.00848.

[21] Y. LECUN. "THE MNIST DATABASE of handwritten digits." In: *http://yann.lecun.com/exdb/mnist/* (). URL: https://cir.nii.ac.jp/crid/1571417126193283840.

[22] Chieh-Hsin Lai, Yang Song, Dongjun Kim, Yuki Mitsufuji, and Stefano Ermon. *The Principles of Diffusion Models*. 2025. arXiv: 2510.21890 [cs.LG]. URL: https://arxiv.org/abs/2510.21890.

[23] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. "Deep Learning Face Attributes in the Wild." In: *Proceedings of International Conference on Computer Vision (ICCV)*. 2015.

[24] Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Rätsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. *Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations*. 2019. arXiv: 1811.12359 [cs.LG]. URL: https://arxiv.org/abs/1811.12359.

[25] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. *Object-Centric Learning with Slot Attention*. 2020. arXiv: 2006.15055 [cs.LG]. URL: https://arxiv.org/abs/2006.15055.

[26] Anita Mahinpei, Justin Clark, Isaac Lage, Finale Doshi-Velez, and Weiwei Pan. *Promises and Pitfalls of Black-Box Concept Learning Models*. 2021. arXiv: 2106.13314 [cs.LG]. URL: https://arxiv.org/abs/2106.13314.

[27] Andrei Margeloiu, Matthew Ashman, Umang Bhatt, Yanzhi Chen, Mateja Jamnik, and Adrian Weller. *Do Concept Bottleneck Models Learn as Intended?* 2021. arXiv: 2105.04289 [cs.LG]. URL: https://arxiv.org/abs/2105.04289.

[28] Loic Matthey, Irina Higgins, Demis Hassabis, and Alexander Lerchner. *dSprites: Disentanglement testing Sprites dataset*. https://github.com/deepmind/dsprites-dataset/. 2017.

[29] Bac Nguyen, Yuhta Takida, Naoki Murata, Chieh-Hsin Lai, Toshimitsu Uesaka, Stefano Ermon, and Yuki Mitsufuji. *Improved Object-Centric Diffusion Learning with Registers and Contrastive Alignment*. 2026. arXiv: 2601.01224 [cs.CV]. URL: https://arxiv.org/abs/2601.01224.

[30] Maxime Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. 2024. arXiv: 2304.07193 [cs.CV]. URL: https://arxiv.org/abs/2304.07193.

[31] Enrico Parisini, Tapabrata Chakraborti, Chris Harbron, Ben D. MacArthur, and Christopher R. S. Banerji. *Leakage and Interpretability in Concept-Based Models*. 2026. arXiv: 2504.14094 [cs.LG]. URL: https://arxiv.org/abs/2504.14094.

[32] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. *FiLM: Visual Reasoning with a General Conditioning Layer*. 2017. arXiv: 1709.07871 [cs.CV]. URL: https://arxiv.org/abs/1709.07871.

[33] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. 2022. arXiv: 2112.10752 [cs.CV]. URL: https://arxiv.org/abs/2112.10752.

[34] Max W. Shen. *Trust in AI: Interpretability is not necessary or sufficient, while black-box interaction is necessary and sufficient*. 2022. arXiv: 2202.05302 [cs.LG]. URL: https://arxiv.org/abs/2202.05302.

[35] Sungbin Shin, Yohan Jo, Sungsoo Ahn, and Namhoon Lee. "A Closer Look at the Intervention Procedure of Concept Bottleneck Models." In: *Workshop on Trustworthy and Socially Responsible Machine Learning, NeurIPS 2022*. 2022. URL: https://openreview.net/forum?id=PUspzfGsgY.

[36] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. 2015. arXiv: 1503.03585 [cs.LG]. URL: https://arxiv.org/abs/1503.03585.

[37] {Casper Kaae} Sønderby, Tapani Raiko, Lars Maaløe, {Søren Kaae} Sønderby, and Ole Winther. "How to Train Deep Variational Autoencoders and Probabilistic Ladder Networks." English. In: *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*. JMLR: Workshop and Conference Proceedings. 33rd International Conference on Machine Learning (ICML 2016), ICML ; Conference date: 19-06-2016 Through 24-06-2016. 2016. URL: http://icml.cc/2016/.

[38] Arash Vahdat and Jan Kautz. *NVAE: A Deep Hierarchical Variational Autoencoder*. 2021. arXiv: 2007.03898 [stat.ML]. URL: https://arxiv.org/abs/2007.03898.

[39] Xin Wang, Hong Chen, Si'ao Tang, Zihao Wu, and Wenwu Zhu. *Disentangled Representation Learning*. 2024. arXiv: 2211.11695 [cs.LG]. URL: https://arxiv.org/abs/2211.11695.

[40] Xinyue Xu, Yi Qin, Lu Mi, Hao Wang, and Xiaomeng Li. *Energy-Based Concept Bottleneck Models: Unifying Prediction, Concept Intervention, and Probabilistic Interpretations*. 2024. arXiv: 2401.14142 [cs.CV]. URL: https://arxiv.org/abs/2401.14142.

[41] Tao Yang, Cuiling Lan, Yan Lu, and Nanning zheng. *Diffusion Model with Cross Attention as an Inductive Bias for Disentanglement*. 2024. arXiv: 2402.09712 [cs.CV]. URL: https://arxiv.org/abs/2402.09712.

[42] Tao Yang, Yuwang Wang, Yan Lv, and Nanning Zheng. *DisDiff: Unsupervised Disentanglement of Diffusion Probabilistic Models*. 2023. arXiv: 2301.13721 [cs.CV]. URL: https://arxiv.org/abs/2301.13721.

[43] Mateo Espinosa Zarlenga et al. *Concept Embedding Models: Beyond the Accuracy-Explainability Trade-Off*. 2022. arXiv: 2209.09056 [cs.LG]. URL: https://arxiv.org/abs/2209.09056.

[44] Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. *Diffusion Transformers with Representation Autoencoders*. 2025. arXiv: 2510.11690 [cs.CV]. URL: https://arxiv.org/abs/2510.11690.